Data Intensive Systems

IN BA6

Notes by Ali EL AZDI

February 16, 2026

Introduction

This document is designed to offer a concise, LaTeX-styled overview of the Data Intensive Systems course, with a focus on essential content and clarity. The **ConceptName.** notation is usually used for definitions in that same spirit. The latest version I am currently working on (not necessarily stable) is available at `https://github.com/elazdi-al/notes`. I do make mistakes often, and sharing this document is also a way to have my notes proofread. If you notice anything weird, incorrect, or missing, please do not hesitate to contact me at ali.elazdi@epfl.ch. I would be happy to answer and explain why I made certain choices.

with this being said, welcome back everyone and good luck!

Contents

Chapter 1

L1 - Introduction

1.1 Why data management?

Modern software is data-driven: systems are useful only when data can be stored, shared, and queried correctly over time. This chapter builds that idea step by step, from raw data to DBMS architecture.

1.1.1 What is data?

We first separate *raw facts* from *usable meaning*.

Data. Raw facts used as input for reasoning, discussion, and computation. Data can be useful, irrelevant, or redundant. It must be processed to become meaningful.

Information. Processed data with meaning in context. Information is relevant to a specific problem and actionable for decision-making.

Practical flow:

$$\text{Data} \xrightarrow{\text{organize + process}} \text{Information}$$

Information then supports higher-level outcomes such as **knowledge** and **wisdom**.

1.1.2 What is a database?

Once data has meaning, we need structure.

Database. A large, integrated, structured collection of data, usually modeling a real-world enterprise.

Entity. An object type we store data about (for example, a student or a course).

Relationship. A meaningful association between entities (for example, a student enrolls in a course).

Example: University. Entities are courses, students, and professors; relationships include enrollment and teaching.

1.1.3 What is a Database Management System (DBMS)?

A database alone is passive. We need software that manages access and correctness.

DBMS. A software system designed to store, manage, and facilitate access to databases.

A useful decomposition is:

$$\text{DBMS} = \text{interrelated data (database)} + \text{programs to access/manage it}$$

1.1.4 What does a DBMS do?

Core role of a DBMS. Provide efficient, reliable, convenient, and safe multi-user storage of and access to massive persistent data.

In practice, this means:

- failure protection against hardware/software faults, power loss, and malicious behavior,
- high throughput (thousands of queries/updates per second),
- high availability (24/7 operation),
- concurrency control for correct multi-user access,
- support for very large-scale data,
- persistence (data outlives programs),
- high-level declarative access with physical data independence.

Persistent data. Data that remains stored beyond the lifetime of a process or program execution.

Concurrency control. Mechanisms that coordinate simultaneous operations so outcomes remain correct.

1.1.5 Is your file system a DBMS?

This thought experiment shows why plain files are often insufficient for data-intensive multi-user applications.

Thought experiment 1: concurrent edits

Two collaborators edit and save the same file at nearly the same time. The natural question is: *whose changes survive*? Outcomes can be yours, your partner's, both, neither, or simply **unclear**.

Thought experiment 2: update during power loss

A file update is in progress and power fails. Which changes survive: all, none, only changes since last save, or again **unclear**?

The key issue is the missing contract: guarantees are unclear, which makes application logic fragile and hard to reason about.

How do you write programs over a platform that promises "???"?

If correctness, durability, isolation, and recovery guarantees are not explicit, every application must re-implement them. **A DBMS exists to provide these guarantees systematically.**

1.1.6 Is the web a DBMS?

The web offers strong information retrieval, but retrieval is not equivalent to database management.

The web is not a DBMS

- data is mostly unstructured and untyped,
- the "correct" answer for many queries is not well-defined,
- completeness is not guaranteed,
- data manipulation guarantees are limited,
- freshness, cross-item consistency, and fault-tolerance guarantees are weaker or undefined.

Practical architecture

Most serious websites place a DBMS behind the web layer (for example, `facebook.com` with MySQL).

Is ChatGPT a DBMS???

Short answer. No. ChatGPT is a language model interface, not a DBMS.

It can connect to database-backed systems, but by itself it does not provide the full DBMS contract (schema-centered storage, transaction guarantees, and complete declarative data management).

1.1.7 The scope of DBMS

The question is no longer only *"where to store bytes"*; it is how to query, update, and trust data under real workload conditions.

What more could we want than a file system?

- simple and efficient *ad-hoc* querying,
- robust concurrency control,
- recovery mechanisms,
- the benefits of well-designed data models.

Ad-hoc query. A query formed for a specific immediate need, not predefined as part of a fixed workflow.

With many queries, users, transactions, and resource constraints, complexity increases quickly. Managing that complexity is a core DBMS concern.

Can the OS offer services like memory management etc? Again, not really

The operating system is essential, but it does not provide the complete data-management contract that applications need.

1.1.8 What is the intellectual contribution?

DBMS is not only engineering practice; it also contributes key ideas about representation, semantics, and correctness.

Core contributions of DBMS thinking

- **Representation of information**: data modeling,
- **Languages and systems for querying data**: complex queries with real semantics over massive data,
- **Concurrency control for data manipulation**: controlled concurrent access with transactional meaning,
- **Reliable data storage**: preserving semantics even after failures (for example, power loss).

Transactional semantics. The guarantee that concurrent and failing executions still behave like correct transactions under clear correctness rules.

1.2 DBMS Architecture

The main architectural question is simple: how do we keep one coherent meaning of data while serving different users and storing bytes efficiently on disk?

1.2.1 Describing data

Data model. A collection of concepts for describing data.

Data models are high-level abstractions: they hide low-level storage details and let us reason about structure and meaning first. Typical model families include *relational*, *hierarchical*, *graph*, and *object-oriented*.

Relation. A named table with rows (tuples) and columns (attributes).

Schema. The structural definition of a relation: attribute names, types, and organization.

In the **relational model**, data is represented as a set of records organized into relations. In **nested models**, not all data is forced into flat tables; hierarchies and arrays are natural first-class structures.

Schema vs. data. Schema is the type (for example, *string*); data is a value instance (for example, *"hello"*) that follows that type.

1.2.2 Example: Schema of a University database

We use one running example throughout the rest of the section.

Students `sid:string`, `name:string`, `login:string`, `age:integer`, `gpa:real`
Enrolled `sid:string`, `cid:string`, `grade:string`
Courses `cid:string`, `cname:string`, `credits:string`

Some later examples use `credits:integer`. The important point is explicit typing in the schema.

1.2.3 Describing data: Levels of abstraction

External schema. User- or application-specific views, often used for access control.

Logical schema (conceptual level). The main global structure seen by users and application programs.

Physical schema (internal level). How data is stored on disk: files, indexes, and storage layout choices.

The mapping is:

$$\text{External schemas} \rightarrow \text{Logical schema} \rightarrow \text{Physical schema}$$

Many external views map to one logical schema that is then implemented by physical structures.

Running example across levels

External views (access control)

Students_Info `sid:string`, `name:string`
Course_Enrollment `cid:string`, `#enrolled:integer`

Logical schema (what applications see)

The logical level uses the university relations above: **Students**, **Courses**, and **Enrolled** (with `credits` often shown as `integer` in this level).

Physical schema (how storage is organized)

Relations can be stored as unordered files, with indexes such as `Students.sid` and `Courses.cid`.

1.2.4 DBMS cares about data independence types

Data independence. The ability to change the schema at one level of a database system without changing the schema at the next higher level.

Logical data independence. The ability to change the conceptual schema without changing user views.

Physical data independence. The ability to change the internal schema without changing the conceptual schema or user views.

This matters because data-intensive systems evolve constantly: applications should keep working while storage layouts, indexes, and conceptual structures improve over time.

1.2.5 Simplified DBMS architecture

The architecture has two linked flows: a *design flow* to define data and a *query flow* to access data efficiently.

Design flow (from requirements to storage)

- **Conceptual design**: capture the enterprise at a high level, typically with ER models.
- **Logical design**: translate ER structures into a relational model.
- **Physical design**: choose indexing and storage structures.
- **Database storage**: persist data in disk files.

Query flow (from SQL to results)

Applications issue SQL (rooted in relational algebra). Inside the DBMS, processing is layered:

- query optimization and execution,
- relational operators,
- access methods,
- buffer management,
- disk space management.

Cross-cutting services enforce correctness and performance across these layers: query processing, transaction management, and concurrency control.

1.2.6 Relational data model

In practice, relational design relies on rows/columns and explicit links between relations through keys.

Key. An attribute (or set of attributes) whose value identifies a tuple uniquely inside a relation.

Foreign key. An attribute (or set) in one relation whose values reference keys in another relation, linking the two relations.

Example relation instance

Enrolled

SID	CID	Grade
53666	Carnatic101	5
53666	Raggae203	5.5
53650	Topology112	6
53666	History105	5

Students

SID	Name	Login	Age	GPA
53666	Jones	jones@cs	18	5.4
53688	Smith	smith@eecs	18	4.2
53650	Smith	smith@math	19	4.8

Here, `Enrolled.SID` acts as a foreign key referencing `Students.SID`.
This raises the next design question: *how do we design such schemas systematically?*

1.2.7 Database design requirements

Database design proceeds in stages, from problem understanding to implementation choices.

- **Requirements analysis**: identify user needs and what the database must support.
- **Conceptual design**: produce a high-level enterprise description (often with ER).
- **Logical design**: translate ER into the DBMS data model.
- **Schema refinement**: improve consistency and normalization.
- **Physical design**: choose indexes and disk layout.
- **Security design**: define who can access what.

1.2.8 What is a conceptual design

Conceptual design. A high-level description of the enterprise data, independent of storage and implementation details.

It answers three foundational questions:

- what are the *entities* and *relationships* in the enterprise?
- what information about them should the database store?
- which *integrity constraints* must always hold?

ER diagram. A pictorial representation of the conceptual schema in the ER model.

An ER diagram can be mapped into a relational schema; this bridge connects conceptual understanding to implementable database design.

1.2.9 Conceptual Design: Entity-Relationship Model

The ER model is the standard language for conceptual design: it captures entities, attributes, relationships, and integrity constraints before logical and physical design decisions.

Basics of ER model

Entity. A real-world object, distinguishable from other objects.

Attribute. A property used to describe an entity in the database.

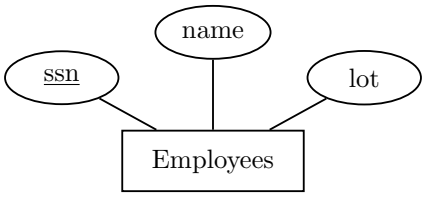
Entity set. A collection of similar entities that share the same attributes.

Domain. The set of allowed values for an attribute.

Key (in ER). An attribute (or minimal attribute set) that uniquely identifies entities in an entity set.

Entity-set schema example

For the entity set **Employees**, we use attributes `ssn`, `name`, and `lot`, with `ssn` as key.



Instance of an entity set. The currently stored collection of entities in that set.

Example instances for **Employees**:

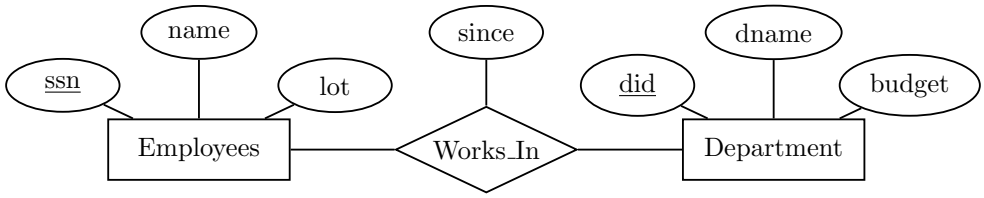
- (`123-22-3666`, `John`, `P1`)
- (`231-31-5368`, `Tom`, `P2`)

Relationships in ER model

Relationship. An association among two or more entities.

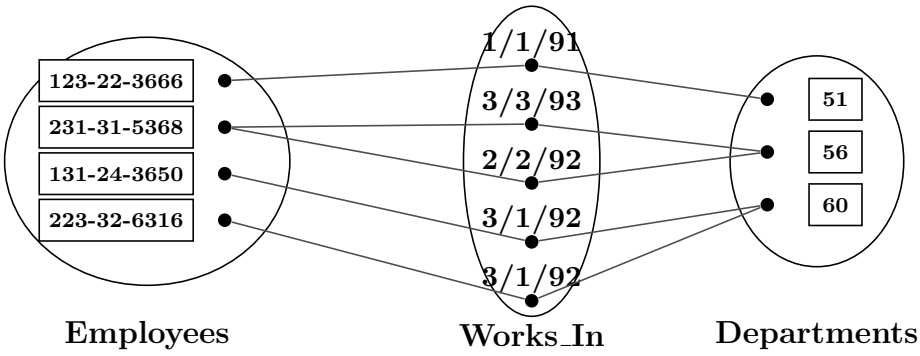
Relationship set. A collection of similar relationships.

Relationships may have their own attributes. Example: **Works_In** links **Employees** and **Department**, with relationship attribute `since`.



Instance of a relationship set. The currently stored set of relationship tuples between participating entity sets.

Example instance of **Works_In**(`ssn`, `did`, `since`) with links to participating entity instances:

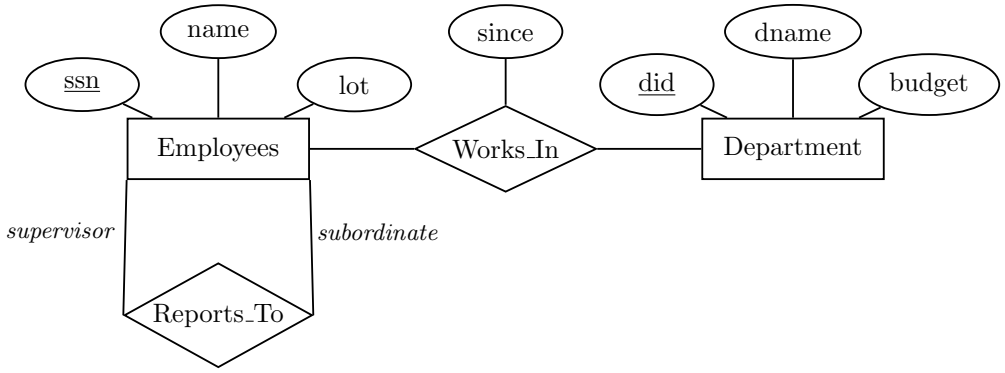


Role names and repeated participation

The same entity set can participate in different relationship sets, and can also appear multiple times in one relationship set under different roles.

Role name. A label identifying the function of an entity set when it appears more than once in the same relationship.

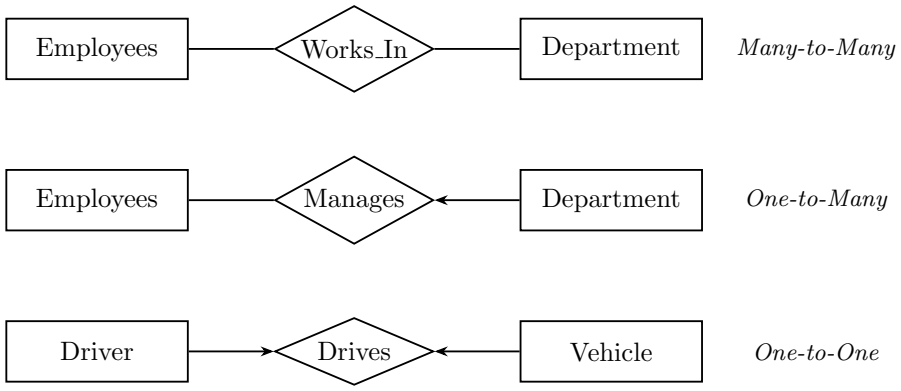
Example: in **Reports_To**, **Employees** appears twice with roles *supervisor* and *subordinate*; the same entity set also participates in other relationships such as **Works_In**.



1.2.10 Key constraints

Key constraints restrict how entities participate in relationships. An arrow from an entity set to a relationship set means each entity participates in *at most one* relationship instance.

Key constraint. A restriction stating that each entity in a set can participate in at most one relationship instance of a given relationship set.



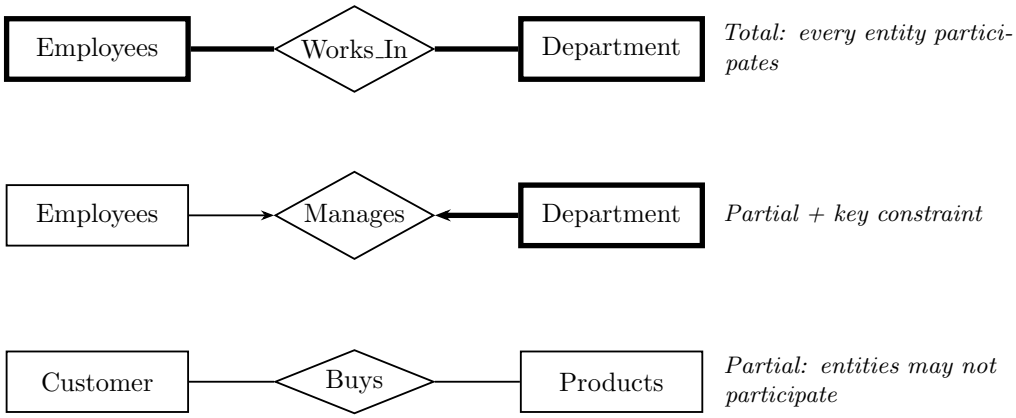
- **Many-to-Many**: no arrows; an employee can work in many departments, and a department can have many employees.
- **One-to-Many**: arrow on one side; each department has at most one manager (key constraint on Manages).
- **One-to-One**: arrows on both sides; each driver drives at most one vehicle and each vehicle has at most one driver.

1.2.11 Participation constraints

Participation constraints specify whether *every* entity in a set must participate in a relationship set.

Total participation. Every entity in the set must participate in at least one relationship instance. Drawn with a **thick line**.

Partial participation. Some entities may not participate in any relationship instance. Drawn with a thin line (default).



Combining key and participation constraints gives precise cardinality semantics:

- thick line + no arrow = total, many: every entity participates, possibly in multiple instances.
- thick line + arrow = total, at most one: every entity participates in exactly one instance.
- thin line + arrow = partial, at most one: an entity participates in zero or one instance.
- thin line + no arrow = partial, many: an entity may or may not participate, with no upper bound.

1.2.12 Weak entities

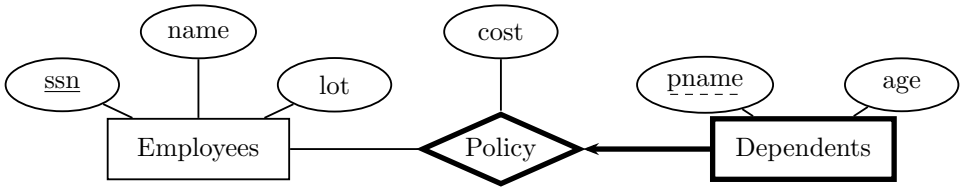
Weak entity. An entity that can be identified uniquely only by considering the primary key of another (*owner*) entity together with its own partial key.

Identifying relationship. The relationship set that links a weak entity set to its owner entity set. The weak entity must have total participation in this relationship.

Rules for weak entities:

- Owner and weak entity set participate in a one-to-many relationship (one owner, many weak entities).
- The weak entity set must have **total participation** in the identifying relationship.
- Weak entities have only a **partial key** (drawn with a dashed underline): it uniquely identifies weak entities only within the scope of a given owner.

Notation: the weak entity and its identifying relationship are drawn with **thick borders**; the arrow from the weak entity to the identifying relationship indicates the key constraint.

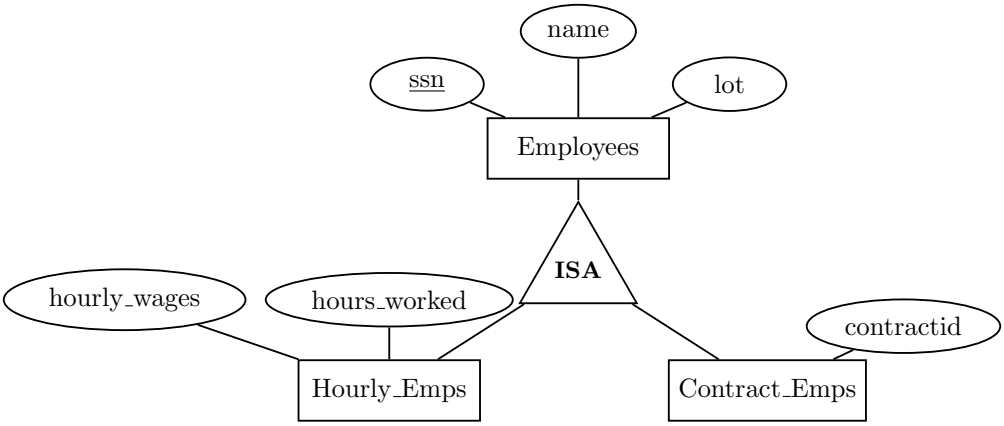


Here, **Dependents** is the weak entity owned by **Employees** through the identifying relationship **Policy**. The attribute `pname` is a partial key.

1.2.13 ISA ("is a") hierarchies

ISA hierarchy. A relationship between a superclass entity set and one or more subclass entity sets, where each subclass entity "is a" member of the superclass. Attributes are inherited from superclass to subclass.

If we declare A **ISA** B, then every A entity is also considered a B entity and inherits all attributes of B.



Hourly_Emps and **Contract_Emps** are subclasses of **Employees**: they inherit `ssn`, `name`, and `lot`, and add their own specialized attributes.

Two orthogonal design constraints govern ISA hierarchies:

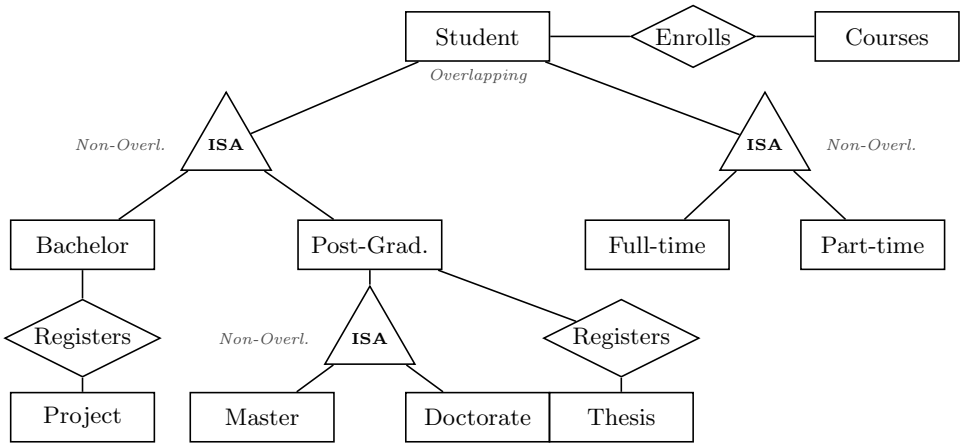
- **Overlap constraint**: can an entity belong to more than one subclass? Default: *no* (disjoint).
- **Covering constraint**: must every superclass entity belong to at least one subclass? Default: *no* (partial coverage).

Reasons to use ISA:

- add descriptive attributes specific to a subclass,
- identify entity sets that participate in specific relationship sets.

ISA example: Student hierarchy

A student can be classified by degree type (Bachelor, Post-Graduate) *and* by enrollment status (Full-time, Part-time) simultaneously. These are two **overlapping** ISA branches from the same superclass. Within each branch, the subclasses are **non-overlapping** (a student is either a Bachelor or a Post-Graduate, not both).



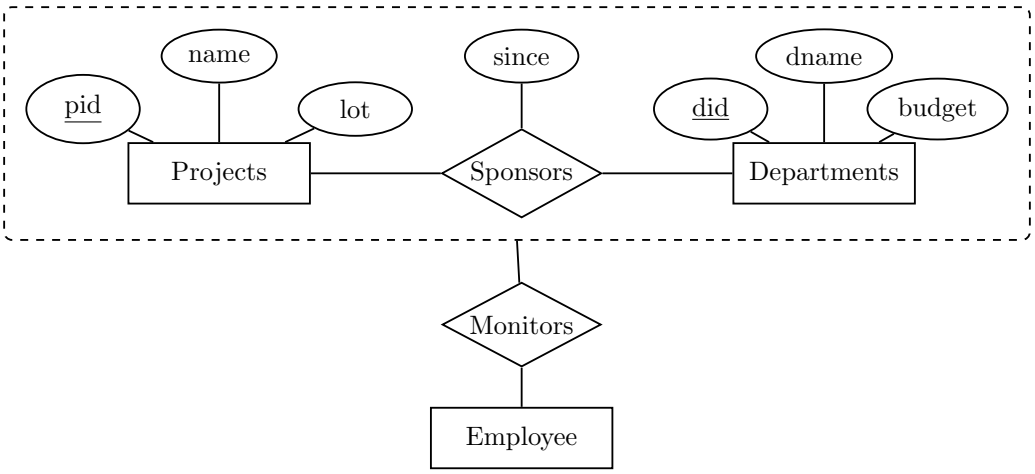
Post-Graduate further specializes into Master and Doctorate (non-overlapping). Each subclass can participate in its own relationship sets: Bachelor students register for Projects; Post-Graduate students register for a Thesis.

1.2.14 Aggregation

Aggregation. A mechanism that treats a relationship set and its participating entity sets as a single higher-level entity set, so that it can participate in other relationships.

Aggregation is needed when we want to model a relationship *involving a relationship set*. For example: departments sponsor projects (the relationship **Sponsors**), and an employee needs to monitor each sponsorship.

Without aggregation we would need a ternary relationship among Projects, Departments, and Employees, which does not capture that monitoring applies to the *sponsorship* itself. With aggregation, the Sponsors relationship is enclosed in a dashed box and treated as an entity set that participates in **Monitors**.



1.2.15 Conceptual design using the ER model

Designing an ER schema involves several recurring choices. The right answer depends on the semantics of the data and the queries the application must support.

Design choices

- Should a concept be modeled as an *entity* or an *attribute*?
- Should a concept be modeled as an *entity* or a *relationship*?
- Should relationships be *binary* or *ternary*? Or use *aggregation*?

Entity vs. attribute

Should *address* be an attribute of Employees or an entity related to Employees?

It depends on how we use the data:

- If there are **several addresses per employee**, address must be an entity (attributes cannot be set-valued in the basic ER model).
- If the **structure** of an address matters (city, street, zip), address must be an entity (attribute values are atomic).

The same reasoning applies to relationship attributes. For example, if **Works_In** has attributes `from` and `to` to record time periods, an employee can work in a department for only one period. If we want to record *several* periods, we must promote the time information into a separate entity (**Duration**) linked by a new relationship.

Entity vs. relationship

Should a concept be a relationship or an entity with relationships to the things it connects? The guideline is: if the concept has **its own identity** or participates in **other relationships**, make it an entity.

Binary vs. ternary relationships

If a ternary relationship can be decomposed into binary relationships without loss of information, prefer binary. If not (some combinations of pairs do not imply the triple), a ternary relationship is needed.

Constraints in the ER model

- A lot of data semantics *can* (and should) be captured in ER diagrams: key constraints, participation constraints, weak entities, ISA hierarchies, and aggregation.
- However, some constraints *cannot* be expressed in ER diagrams (for example, "a manager must also be an employee of the managed department"). These must be enforced by the application or by database-level integrity constraints.

Chapter 2

L2 - Relational Model & SQL Basics

2.1 Relational model

The relational model is the foundation of modern database systems. It represents all data as tables and provides a clean, mathematically grounded framework for querying and constraining data.

2.1.1 Core concepts

A **database** is a set of named *relations* (*named tables with rows and columns*). Each attribute carries a **type** (domain): integer, real, string, file formats, enumerated, etc.

Tuple. A single row in a relation, containing one value for each attribute. All tuples in a relation are distinct and have no inherent order.

Running example:

Students

sid	name	login	age	gpa
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smit@ee	18	3.2
...	...	...	...	...

Colleges

name	location	strength
MIT	USA	10000
Oxford	UK	22000
EPFL	CH	9000
...	...	...

2.1.2 Schema vs. instance

A relation has two complementary aspects: its *schema* (*structural definition with attribute names and types*) and its content at a given moment.

Example schema: `Students(sid: string, name: string, login: string, age: integer, gpa: real)`.

Instance. The actual set of tuples stored in a relation at a given point in time.

Two measures describe an instance:

- **Cardinality**: the number of tuples (rows).
- **Arity** (or degree): the number of attributes (columns).

2.1.3 Null values

Not every attribute value is always known. A student who has not received any grades yet has no meaningful GPA.

Null value. A special marker representing an "unknown" or "undefined" attribute value. Nulls are not the same as zero or an empty string.

2.2 Keys and integrity constraints

Keys (*attributes that uniquely identify tuples*) enforce uniqueness and link relations together. Here we refine the hierarchy of key types and formalize integrity constraints.

2.2.1 Key hierarchy

Superkey. A set of attributes such that no two distinct tuples can share the same values in all those fields. Any superset of a key is also a superkey.

Candidate key. A *minimal* superkey: it is a superkey, and no proper subset of its fields is also a superkey.

Primary key. The candidate key chosen by the database administrator (DBA) as *the* key of the relation.

Example with **Students**(sid, name, login, age, gpa): both `sid` and `login` are candidate keys. The DBA picks `sid` as the primary key.

- Every superset of `sid` (e.g., {`sid, name`}, {`sid, login, gpa`}, ...) is a superkey but not a candidate key.
- `sid` alone and `login` alone are the only candidate keys.

2.2.2 Integrity constraints

A key is an **integrity constraint** (IC): a condition that must hold on every valid database instance.

Foreign keys (*attributes referencing the primary key of another relation*) establish **referential integrity**: every foreign-key value must match an existing key in the referenced relation.

2.3 SQL: Structured Query Language

SQL is the standard language for interacting with relational databases. It splits into two sub-languages.

2.3.1 DDL and DML

Data Definition Language (DDL). The subset of SQL used to create, modify, and delete relations, specify constraints, and administer users and security.

Data Manipulation Language (DML). The subset of SQL used to query, insert, update, and delete tuples.

2.3.2 Most common SQL commands

```
CREATE TABLE <name> (<field> <domain>, ...)
INSERT INTO <name> (<field names>) VALUES (<field values>)
DELETE FROM <name> WHERE <condition>
UPDATE <name> SET <field name> = <value> WHERE <condition>
SELECT <fields> FROM <name> WHERE <condition>
```

2.3.3 Creating relations

A `CREATE TABLE` statement defines a new relation by listing each attribute and its domain. The type of each field is enforced by the DBMS whenever tuples are added or modified.

Students – holds data about students:

```
CREATE TABLE Students
  (sid CHAR(20),
   name CHAR(20),
   login CHAR(10),
   age INTEGER,
   gpa FLOAT)
```

Enrolled – holds course enrollments:

```
CREATE TABLE Enrolled
  (sid CHAR(20),
   cid CHAR(20),
   grade CHAR(2))
```

2.3.4 Adding and deleting tuples

Insert a single tuple:

```
INSERT INTO Students (sid, name, login, age, gpa)
VALUES ('53688', 'Smith', 'smith@cs', 18, 3.2)
```

Delete all tuples satisfying a condition:

```
DELETE FROM Students S WHERE S.name = 'Smith'
```

2.3.5 Keys in SQL

The `PRIMARY KEY` clause declares the primary key; `UNIQUE` marks additional candidate keys.

```
CREATE TABLE Students
  (sid CHAR(20),
   name CHAR(20),
   login CHAR(10),
   age INTEGER,
   gpa FLOAT,
   PRIMARY KEY(sid))
```

```
CREATE TABLE Person
  (ssn CHAR(9),
   name CHAR(20),
   licence# CHAR(10),
   PRIMARY KEY(ssn),
   UNIQUE(licence#))
```

Choosing keys carefully

Keys must be used carefully: they encode real-world constraints. Consider `Enrolled`: "for a given student and course, there is a single grade."

Correct – composite PK on `(sid, cid)`:

```
CREATE TABLE Enrolled
  (sid CHAR(20),
   cid CHAR(20),
   grade CHAR(2),
   PRIMARY KEY(sid, cid))
```

Wrong – PK on `sid` alone with `UNIQUE(cid, grade)` encodes a different constraint: "students can take only one course, and no two students in a course receive the same grade."

2.3.6 Foreign keys and referential integrity

Foreign keys (*fields referencing the primary key of another relation*) act as logical pointers between relations. If all foreign-key constraints are enforced, the database achieves **referential integrity** (no dangling references).

Students

sid	name	login	age	gpa
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smit@ee	18	3.2
...	...	...	...	...

Enrolled

cid	sid	grade
Carnatic101	53666	C
Reggae203	50000	B
Topology112	53666	A
...	...	...

Here `Enrolled.sid` is a foreign key referencing `Students.sid`: every enrolled student must actually exist in `Students`.

Enforcing referential integrity

The DBMS must decide what to do when a foreign-key constraint is threatened:

- **Insert** an Enrolled tuple with a non-existent `sid` → reject the insert.
- **Delete** a Students tuple that is referenced by Enrolled → several policies:
  - cascade-delete all Enrolled tuples that refer to it,
  - disallow the deletion entirely,
  - set the referencing `sid` to a default value,
  - set it to `NULL` (denoting "unknown" or "inapplicable").
- **Update** the primary key of a Students tuple → same policies apply.

2.3.7 Integrity constraints

Integrity constraint (IC). A condition that must be true for *any* instance of the database (e.g., domain constraints, key constraints, foreign-key constraints). ICs are specified when the schema is defined and checked whenever relations are modified.

Legal instance. An instance of a relation that satisfies all specified ICs. The DBMS should not allow illegal instances.

When the DBMS enforces ICs, stored data is more faithful to real-world meaning and data-entry errors are caught early.

2.4 Relational query languages

Query language (QL). A language that allows manipulation and retrieval of data from a database. Unlike programming languages, QLs are not Turing-complete: they are designed for easy, efficient access to large data sets, not general computation.

The relational model supports simple yet powerful QLs with a strong formal foundation based on logic, enabling significant optimization.

2.4.1 Formal relational query languages

Two mathematical query languages form the basis for real languages like SQL:

Relational Algebra. An *operational* language: it specifies *how* to compute a result step by step. Useful for representing execution plans inside database engines.

Relational Calculus. A *declarative* language: it specifies *what* the result should look like, not how to compute it. Non-procedural.

Codd's Theorem. Relational algebra and relational calculus are equivalent in expressive power: for every query in one, there is an equivalent query in the other.

Understanding both is key to understanding SQL and query processing.

2.4.2 Preliminaries

A query is applied to *relation instances*, and the result of a query is also a relation instance.

- The *schemas* of input relations are fixed (but the query runs over any legal instance).
- The schema of the *result* is also fixed, determined by the query language constructs.

Two notations are used:

- **Positional notation**: easier for formal definitions.
- **Named-field notation**: more readable; both are used in SQL.

2.4.3 Example schema and instances

We use a university database throughout the relational algebra examples:

```
instructor  ID, name, dept_name, salary
course      course_id, title, dept_name, credits
teaches     ID, course_id, sec_id, semester, year
section     course_id, sec_id, semester, year, building, room_number, time_slot_id
student     ID, name, dept_name, tot_cred
takes       ID, course_id, sec_id, semester, year, grade
```

Running example instance of `instructor`:

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

2.4.4 Five basic operations

Relational algebra has five primitive operators, split into two groups:

Unary operators (one input relation):

- **Selection** (σ): output a subset of *rows* (horizontal filtering).
- **Projection** (π): output a subset of *columns* (vertical filtering).

Binary operators (two input relations):

- **Cross-product** ($\times$): concatenate every row of the first relation with every row of the second.
- **Set-difference** ($-$): output tuples in R that are not in S.
- **Union** ($\cup$): output tuples in R or in S (or both).

Since each operation returns a relation, **operations can be composed**.

2.4.5 Selection (σ)

Selection outputs only the rows that satisfy a given condition. The output schema is the same as the input.

$$\sigma_{\text{condition}}(relation)$$

Without condition

The simplest expression $\sigma(instructor)$ returns all rows unchanged (equivalent to `SELECT * FROM instructor`).

With a single condition

$$\sigma_{dept\_name=\text{“Physics”}}(instructor)$$

```
SELECT * FROM instructor WHERE dept_name = 'Physics'
```

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

With a compound condition

Conditions can be combined with $\wedge$ (and), $\vee$ (or), and $\neg$ (not):

$$\sigma_{dept\_name=\text{“Physics”} \wedge salary>90000}(instructor)$$

```
SELECT * FROM instructor WHERE dept_name = 'Physics' AND salary > 90000
```

ID	name	dept_name	salary
22222	Einstein	Physics	95000

2.4.6 Projection (π)

Projection retains only the attributes in the projection list. The output schema contains exactly those attributes.

$$\pi_{\text{attribute list}}(relation)$$

$$\pi_{ID,\ name,\ salary}(instructor)$$

corresponds to:

```
SELECT ID, name, salary FROM instructor
```

The result has three columns instead of four (`dept_name` is dropped), but all 12 rows are kept because each row is still distinct.

Duplicate elimination

In relational algebra, projection **eliminates duplicates** because the result must be a set of tuples.

$$\pi_{dept\_name}(instructor)$$

reduces the 12 rows to 7 distinct department names. In SQL, duplicates are kept by default; to match the algebra, use `SELECT DISTINCT`:

```
SELECT DISTINCT dept_name FROM instructor
```

2.4.7 Composing operators

Since every operator returns a relation, the output of one can feed into another. Selection followed by projection gives specific columns from filtered rows:

$$\pi_{name}(\sigma_{dept\_name=\text{“Physics”}}(instructor))$$

```
SELECT name FROM instructor WHERE dept_name = 'Physics'
```

name
Einstein
Gold

First σ filters to Physics instructors, then π keeps only the `name` column. The order matters: projecting first would discard the column needed for selection.

2.4.8 Rename operator (ρ)

The rename operator (ρ) renames a relation or its attributes. Since algebraic expressions can yield unnamed relations natively, renaming is essential to avoid naming conflicts or to self-reference.

- Renames the list of attributes in the form oldname $\rightarrow$ newname or position $\rightarrow$ newname.
- Can be used to rename the output relation itself entirely.
- **Output schema**: same as input except for the renamed attributes.
- **Output instances**: returns the exact same tuples as the input.

Renaming the relation itself to i:

$$\rho_i(instructor)$$

Renaming the ID attribute to $instructor.ID$:

$$\rho_{ID \rightarrow instructor.ID}(instructor)$$

In SQL, this is achieved using the `AS` keyword in the `SELECT` or `FROM` clauses:

```
SELECT i.name FROM instructor AS i WHERE i.ID = ...
```

2.4.9 Union operator ($\cup$)

The union operator takes two input relations, which **must** be *union-compatible*:

- They must have the same number of fields.
- The "corresponding" fields must have exactly the same type.

Like projection, union in strict relational algebra implies *duplicate elimination*.

$$c1 \cup c2$$

In SQL, `UNION` implicitly eliminates duplicates, while `UNION ALL` is faster but keeps them.

```
(SELECT * FROM c1)
UNION
(SELECT * FROM c2)
```

2.4.10 Set-difference operator ($-$)

The set-difference operator outputs the set of tuples in $c1$ which are not in $c2$.

- The two input relations must also be *union-compatible*.
- Set difference is **not** commutative ($c1 - c2 \neq c2 - c1$).

$$c1 - c2$$

In SQL, this corresponds to the `EXCEPT` clause:

```
(SELECT * FROM c1)
EXCEPT
(SELECT * FROM c2)
```

2.4.11 Compound operators

Alongside the five basic operators ($\sigma, \pi, \times, -, \cup$), there are several additional **compound operators**. These add **no computational power** to the language (they can all be expressed using the basic operators), but they serve as useful, compact shorthands.

Intersection ($\cap$)

Intersection outputs the overlapping tuples from two *union-compatible* relations.

$$R \cap S = R - (R - S)$$

```
(SELECT * FROM c1)
INTERSECT
(SELECT * FROM c2)
```

2.4.12 Cross-product operator ($\times$)

The Cartesian product $S \times R$ pairs every single row of S with every single row of R.

- **Result schema**: has one field per field of S and R, inheriting the field names linearly.
- **Naming conflicts**: Both S and R may have fields sharing the exact same name. In this case, we use the renaming operator (ρ) to disambiguate.

$$instructor \times teaches$$

2.4.13 Join operator ($\bowtie$)

A simple $instructor \times teaches$ associates *every* instructor tuple with *every* teaches tuple. Most of these rows hold information about an instructor paired with a course they never taught!

To get only tuples of instructors and the specific courses they actually taught, we apply a selection:

$$\sigma_{instructor.id=teaches.id}(instructor \times teaches)$$

Because this pattern (σ after $\times$) is overwhelmingly common in databases, it gets its own dedicated operator called a **Join**.

$$instructor \bowtie_{instructor.id=teaches.id} teaches$$

If the matched columns have the identical name in both tables, the condition can be completely omitted. This is called a **Natural Join**.

In SQL, a join is explicitly specified using `JOIN`:

```
SELECT * FROM instructor
JOIN teaches ON instructor.ID = teaches.ID
```

2.4.14 Complex queries

Operators can be chained to answer sophisticated questions. Consider: "Find the names of all instructors in the Music department together with the course title of all the courses that they teach."

This query involves three relations (`instructor`, `teaches`, `course`) and requires joining them before projecting the desired columns.

Two equivalent formulations:

(a) Apply the selection *after* the full join:

$$\pi_{name,\ title}(\sigma_{dept\_name=\text{“Music”}}(instructor \bowtie (teaches \bowtie \pi_{course\_id,\ title}(course))))$$

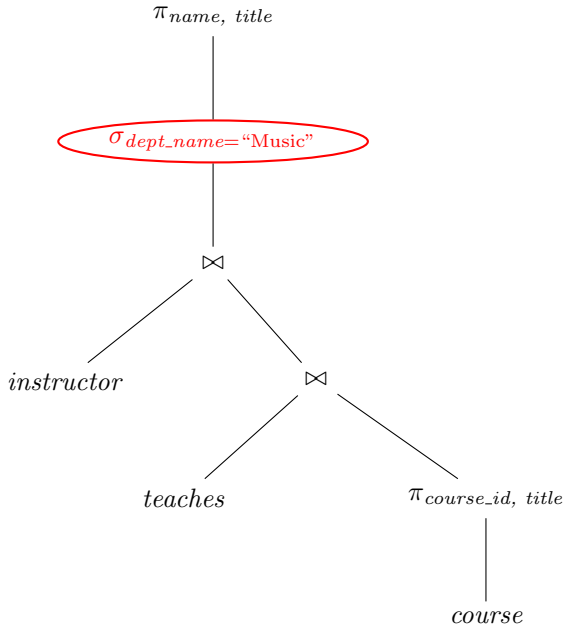
(b) Push the selection *before* the join:

$$\pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”}}(instructor)) \bowtie (teaches \bowtie \pi_{course\_id,\ title}(course)))$$

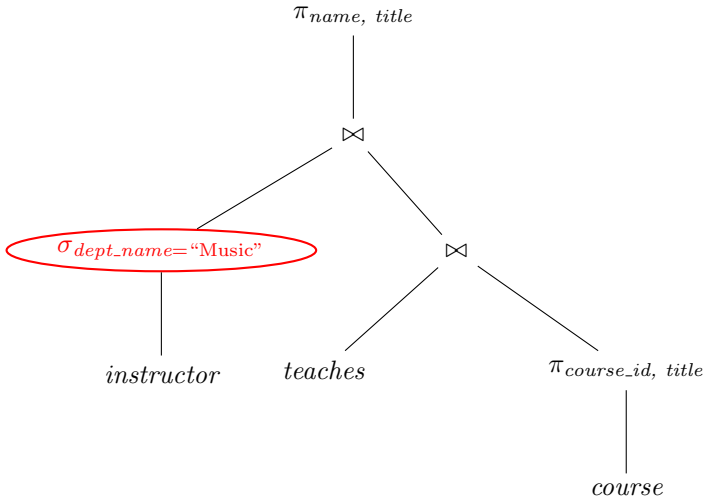
Both return the same result, but formulation (b) is significantly more efficient: the selection filters the instructor relation *before* the expensive join, drastically reducing the number of intermediate tuples.

Query optimization

This illustrates a key principle: **query optimization determines performance**. The DBMS query optimizer automatically rewrites expressions (e.g. pushing selections down) to find an efficient execution plan. The two expression trees below show the difference visually:



(a) Initial: σ applied *after* joining all three relations — processes every tuple.



(b) Optimized: σ pushed *down* to filter *instructor* before the join — far fewer tuples flow upward.

Both trees compute the same result, but (b) is far more efficient because the expensive join operates on a tiny filtered relation instead of the full table.

2.4.15 Division operator (/)

Division is a compound operator useful for expressing "for all" queries (e.g., "find all suppliers who supply *every* part").

Division (A/B). Given relations $A(x, y)$ and $B(y)$, the division A/B returns all values of x such that for **every** tuple b in B, the tuple (x, b) exists in A.

Worked example

Consider relation $A(\texttt{sno}, \texttt{pno})$ and three divisors $B_1 \subseteq B_2 \subseteq B_3$:

$B_1 = \{\text{p2}\}$ $B_2 = \{\text{p2, p4}\}$ $B_3 = \{\text{p1, p2, p4}\}$

$A/B_1 = \{\text{s1, s2, s3, s4}\}$ $A/B_2 = \{\text{s1, s4}\}$ $A/B_3 = \{\text{s1}\}$

A

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

As B grows (more conditions to satisfy), the result of A/B shrinks: only suppliers matching *every* part in B survive.

Expressing division with basic operators

Division can be expressed using only π, $\times$, and $-$.

Idea: find all x values that are *not* "disqualified". An x is disqualified if pairing it with some $y \in B$ yields a tuple *not* in A.

Disqualified x values:

$$\pi_x((\pi_x(A) \times B) - A)$$

Final result:

$$A/B \;=\; \pi_x(A) \;-\; \pi_x((\pi_x(A) \times B) - A)$$

Step by step with $B_3 = \{\textbf{p1, p2, p4}\}$

- $\pi_{sno}(A) = \{\text{s1, s2, s3, s4}\}$
- $\pi_{sno}(A) \times B_3$ produces all $4 \times 3 = 12$ possible (sno, pno) pairs.
- $(\pi_{sno}(A) \times B_3) - A$ keeps only the *missing* pairs: (s2, p4), (s3, p1), (s3, p4), (s4, p1).
- Projecting on `sno` gives the disqualified suppliers: {s2, s3, s4}.
- $\pi_{sno}(A) - \{\text{s2, s3, s4}\} = \{\text{s1}\}$.

Chapter 3

L3 - Storage, Files, and Indexing

Extra: SQL in the era of AI

Large language models (LLMs) make one question unavoidable: will natural language (NL) replace SQL? Will people soon query databases in plain English?

NL2SQL systems. AI systems that turn natural language into SQL queries. Examples include Palimpzest and Lotus. They act as a *front-end*: they help users ask, draft, and explain queries, but they do not replace the database itself.

Databases and LLMs play different roles:

- **DBMS (Back-end)**: Stores data, enforces constraints, manages transactions, controls access, and executes queries efficiently.
- **LLM (Front-end)**: Turns prompts into text or code, but gives **no correctness guarantees**.

Current limitations of NL2SQL

These tools are useful, but they still need careful checking:

- People still outperform the best current systems (for example, 93% vs. 82% on the BIRD benchmark).
- Execution accuracy drops significantly as query difficulty increases.
- Common AI errors include wrong joins, wrong aggregation level, missing `DISTINCT`, and wrong filter order.

Why relational reasoning still matters

SQL and the relational model give **precision, composability, and optimization**. Even if natural language becomes the visible interface, the database still runs on these ideas.

If you do not understand SQL and relational reasoning, you cannot:

- **Verify** the correctness of AI-generated queries.
- **Debug** query failures or logical errors.
- **Predict** and optimize database performance.

3.1 File and access layer

At the physical level, a database is a **file of records**. The DBMS stores these records in operating-system files and supports a small set of access operations.

The main operations at this layer are:

- create and delete files,
- insert, delete, and update records,
- **point access**: retrieve one record via its identifier,
- **range access**: retrieve all records satisfying a condition,
- **scan**: read the entire file.

3.1.1 File organization and indexing

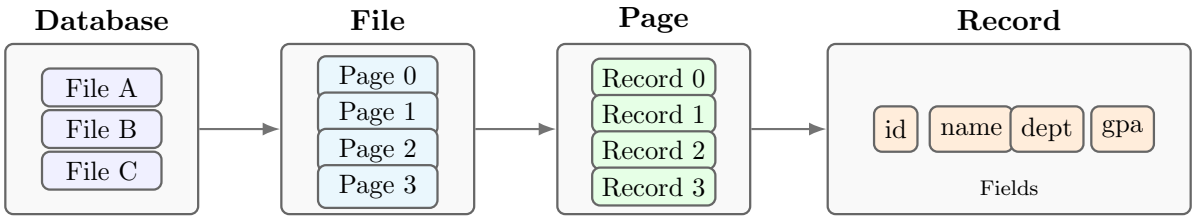
This layer answers three questions:

- how records are stored in files,
- how indexes speed up access,
- how pages are placed on storage.

Metadata about these choices is stored in the *system catalog*.

Storage hierarchy. Database storage has levels:

- a **file** is a set of **pages** (blocks),
- a **page** stores multiple **records**,
- a **record** is a sequence of **fields**.



Physical design means choosing a file organization, managing free space, and adding indexes for fast access.

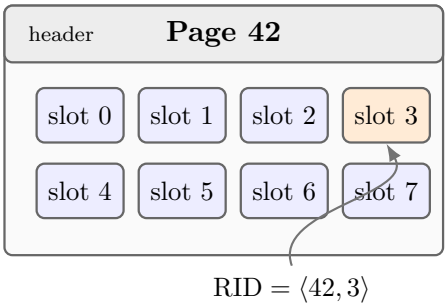
3.2 Page and record formats

Most databases use the N-ary storage model (row store): each page stores many complete tuples, one per slot.

Record Identifier (RID). A record is usually identified by its page id and slot number, often written $\langle \text{page\_id}, \text{slot\_number} \rangle$. Some systems show a logical identifier instead, but it still maps to a page and a slot.

A page format must support search, insertion, and deletion efficiently. The first question is whether records are **fixed-length** or **variable-length**.

Once the DBMS finds the right page, the slot number points to the record inside that page. **Keeping this identifier stable is one of the main goals of page design.**



3.2.1 Fixed-length records

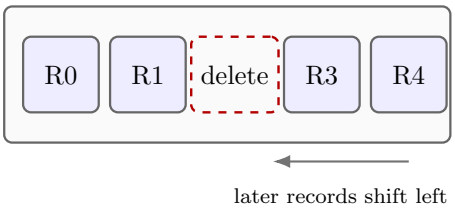
If all fields have fixed size, for example integers or fixed-size `char` values, then every record has the same length:

- the schema is fixed and stored in the catalog,
- field i starts at a fixed offset,
- pages can use very simple slot layouts.

Fixed-length page layouts

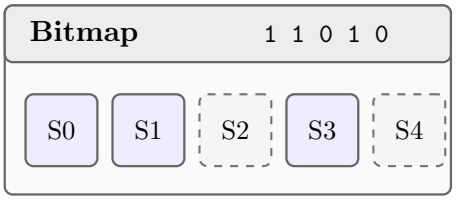
Packed page

Contiguous records



Records are stored next to each other, so space usage is very good. The downside is that deleting one record shifts later records and changes their RIDs.

Bitmap page



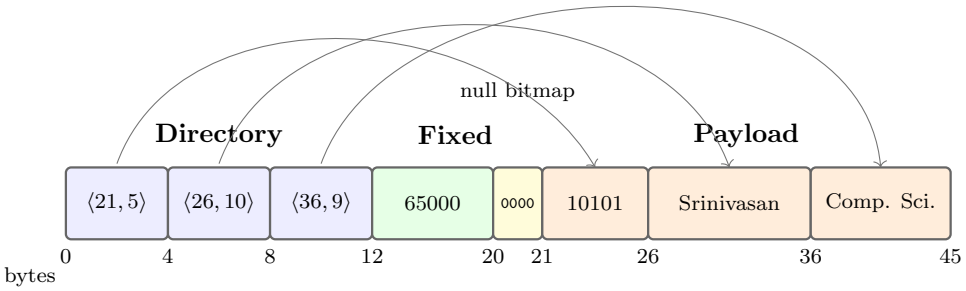
A bitmap shows which fixed slots are in use. When a record is deleted, its bit is cleared. The empty slot can be reused later, and record positions stay stable.

3.2.2 Variable-length records

We need variable-length layouts when records contain fields such as `varchar`, or when one file stores several record types.

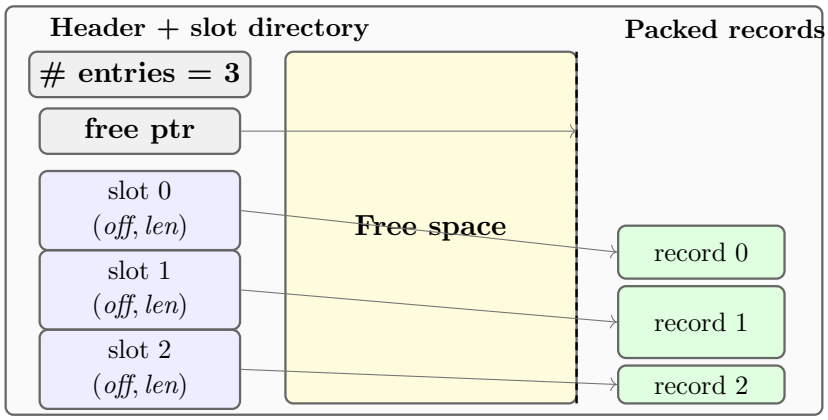
A common layout contains:

- fixed-size values at the front,
- one $\langle \text{offset}, \text{length} \rangle$ entry per variable field,
- the actual variable-length data at the end,
- a small null bitmap for missing attributes.



Slotted page structure

To store variable-length records, most systems use a **slotted page**. The slot number stays fixed even if the record itself moves within the page.



The header and slot directory grow from the front of the page. The records grow from the back. A free-space pointer marks the boundary between them. During compaction, the DBMS can move the record bytes without changing the slot number, so the RID stays the same.

The header stores:

- the number of slot entries,
- a free-space pointer,
- one slot entry per record with its location and size.

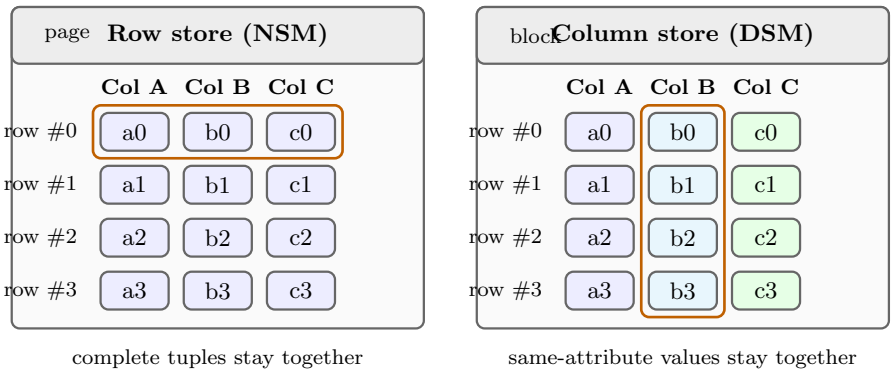
Operational issues

- **record growth**: updating a field may force data to shift within the page,
- **page overflow**: if the record no longer fits, it may move to another page, often with a forwarding pointer,
- **oversized records**: some systems forbid records larger than a page, while others spill large values to overflow storage.

3.3 Row- and column-oriented layouts

The page layouts above use the classic row-store design: each slot points to one full tuple. This is also called the **N-ary storage model (NSM)**.

Decomposition storage model (DSM). A column-oriented layout where each attribute is stored separately. To rebuild a tuple, the system reads the values at the same logical position from the needed columns.



3.3.1 Row store versus column store

Row stores keep all attributes of a tuple together. They work well when queries read or update whole records.

- **row-store advantages**: fast inserts, updates, and deletes; good when workloads read or rewrite full tuples; works well with clustered, index-based storage.
- **row-store disadvantages**: inefficient when a query scans many tuples but needs only a few attributes; compression is weaker because one page mixes many value types.

Column stores make the opposite trade-off. They optimize scans over attributes instead of full tuples.

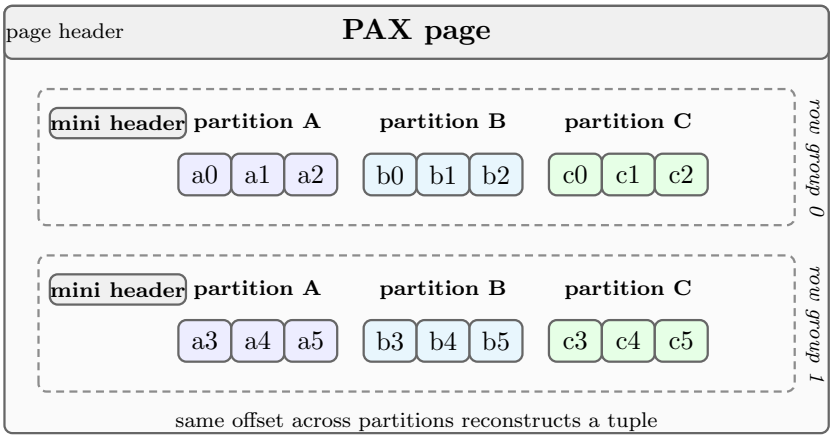
- **column-store advantages**: reduced I/O when only some attributes are needed, better CPU cache locality, stronger compression, and efficient vectorized processing.
- **column-store disadvantages**: full-tuple reconstruction is expensive, updates and deletions are harder, and compressed data may need decompression before processing.

As a rule of thumb, **row stores** fit transactional workloads (**OLTP**), while **column stores** fit analytical workloads (**OLAP**). Some systems support both and act as **hybrid row/column stores**.

3.3.2 Partition Attributes Across (PAX)

Pure column stores separate attributes across the whole file. **PAX** is a middle ground: from the outside the page still looks like a row-store page, but inside the page the values are grouped by column.

PAX. Within one page, values of the same attribute are stored together in column partitions. A tuple is rebuilt logically by taking the same offset in each partition.



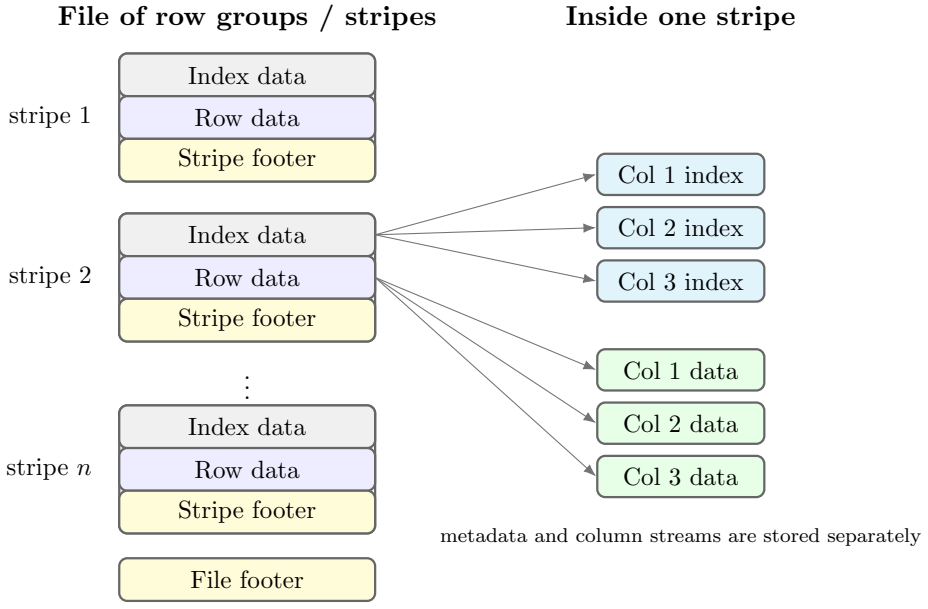
A PAX page keeps:

- the same number of tuples per page as a traditional row page,
- one contiguous partition per attribute inside the page,
- logical rather than physical row boundaries.

This improves cache behavior for scans over a few attributes without giving up the page-based design already used by row stores.

3.3.3 Columnar file formats

Modern analytical file formats apply the same idea at the file level. Formats such as **Parquet** and **ORC** group tuples into large row groups (or stripes) and store each group column by column.



These formats are popular in big-data systems because:

- each column stream compresses well,

- the engine can skip irrelevant columns and often entire groups,
- scans over a subset of attributes avoid touching the rest of the file.

3.4 Alternative file organizations

So far, we looked at how records are stored *inside* pages. Another design choice is how pages are arranged *inside* a file.

- **heap files**: good when typical access is a full file scan or when insertion simplicity matters most,
- **sorted files**: good when records are often read in order or by key range,
- **index-based organizations**: discussed in the indexing part.

3.4.1 Heap files

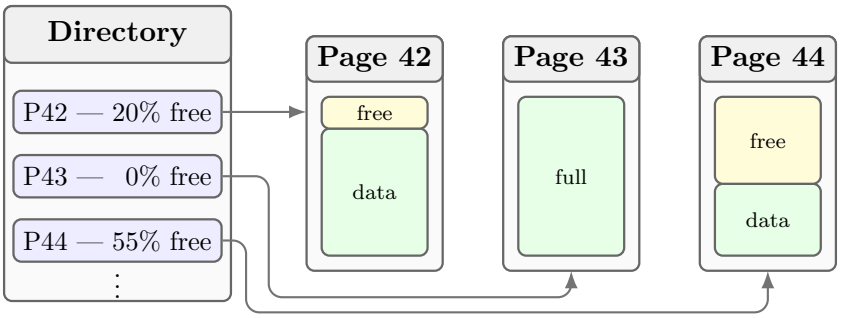
Heap file. An unordered file organization: records are stored in no particular search order.

Heap files are the simplest file organization.

- records can be scanned sequentially, or accessed directly if their RID is known,
- inserts place a new record in any page with enough free space,
- as the file grows or shrinks, pages are allocated and deallocated, so the DBMS must track free space.

The DBMS usually finds heap pages in one of two ways:

- **linked lists**: a header page points to a list of free pages and a list of data pages,
- **page directories**: directory pages store each data page location together with its remaining free space.



directory entry = page id + free-space summary

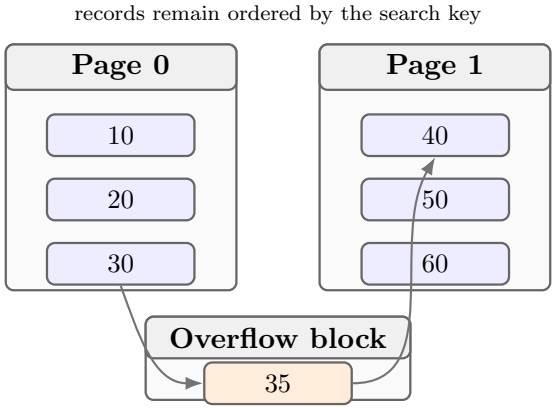
Heap files work well when most queries scan the file, because the DBMS does not maintain physical order.

3.4.2 Sorted (sequential) files

Search key. Attributes used to physically order records in a sorted file. They may be unique, but they do not have to be.

In a sorted file, records appear in search-key order.

- this is good for scanning the whole file in order,
- range queries are efficient because, after the first match, the DBMS can keep reading forward in order.



Keeping that order makes updates more expensive:

- **deletion**: systems often use pointer chains or tombstones so later records do not need to move right away,
- **insertion**: the DBMS finds the right position; if the target page has room, it inserts there, otherwise it uses an overflow page and updates the chain,
- over time, overflow pages accumulate, so the file must sometimes be reorganized to restore physical order.

3.4.3 Heap files versus sorted files

Which organization is better depends on the workload. Heap files make updates cheap. Sorted files make ordered search cheap. To compare them, we use a simple cost model.

Simplified I/O cost model. Measure cost only in page I/Os. In this model, I/O dominates CPU work, so we ignore buffering, prefetching, and other system effects. We also focus only on the average case.

Assumptions:

- B is the number of data pages in the file.
- Each operation inserts or deletes a single record.
- An equality search returns exactly one matching record.
- In a heap file, insertion appends at the end of the file.
- In a sorted file, searches use the file-ordering attribute and deletions keep the file compacted.

The main trade-off is:

- scans cost about the same in both organizations,
- sorted files are much better for equality and range search on the ordering attribute,
- heap files are much better for inserts and usually for deletes, because they avoid shifting many records.

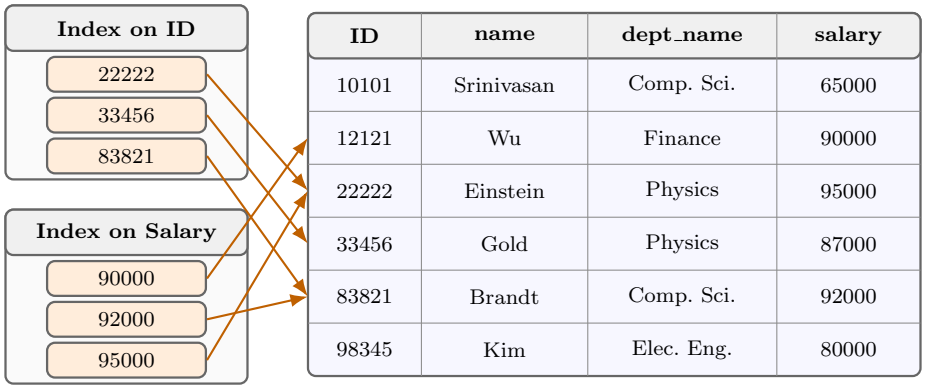
A sorted file gives one very good access path, but it pays heavily for updates. This is why indexes are useful.

Operation	Heap file	Sorted file	Notes
Scan all records	B	B	$B = \#$ of data pages
Equality search	$0.5B$	$\log_2 B$	assumes exactly one matching record
Range search	B	$\log_2 B + \#\text{match pages}$	after the first match, keep reading forward
Insert	2	$\log_2 B + 2 \cdot (B/2)$	must read and rewrite about half the file
Delete	$0.5B + 1$	$\log_2 B + 2 \cdot (B/2)$	search first, then pay the same shifting cost

3.5 Indexing

A sorted file helps only for one physical order. If the file is sorted on `ID`, then lookups on `ID` are efficient, but predicates on `salary` or `dept_name` may still need a scan.

Index. An auxiliary access structure that maps search-key values to records or pages, so the DBMS can answer selections with fewer I/Os than a full file scan.



one physical order, multiple efficient access paths

Indexes matter because:

- they give the same relation several efficient access paths,
- they speed up selections on the index's search-key fields,
- the search key does not need to be unique,
- any subset of relation attributes can be used as a search key,
- inserts, deletes, and updates must also maintain the affected indexes.

3.5.1 Index search conditions

Index search condition. A predicate of the form $\langle \text{search key}, \text{comparison operator} \rangle$.

Examples:

- `dept_name = "CS"`: search key = `dept_name`, operator = equality.
- `gpa > 3`: search key = `gpa`, operator = greater-than.

Equality predicates ask for one exact key value. Inequality predicates such as $<$, $\leq$, $>$, and $\geq$ define ranges, so they benefit most from ordered access paths.

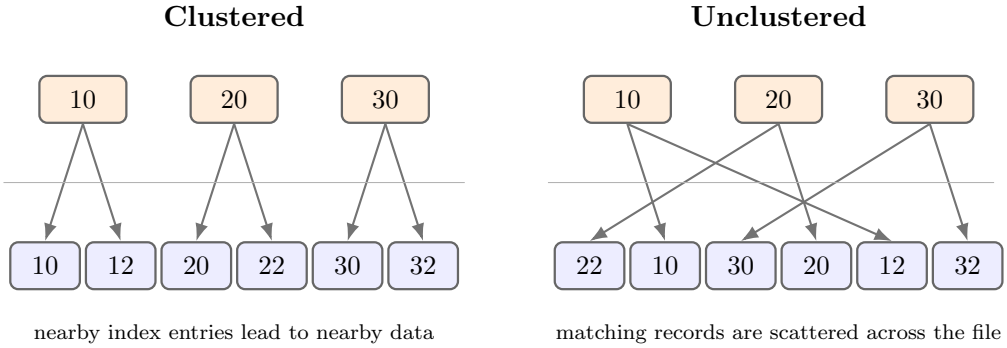
3.6 Index classification

Indexes are commonly described along four independent axes:

- **clustered vs. unclustered**,
- **dense vs. sparse**,
- **key vs. non-key search key**,
- **data file ordered vs. not ordered on the indexing field**.

3.6.1 Clustered and unclustered indexes

Clustered vs. unclustered. If the physical order of the data records follows the order of the index entries, the index is **clustered**. Otherwise it is **unclustered**.

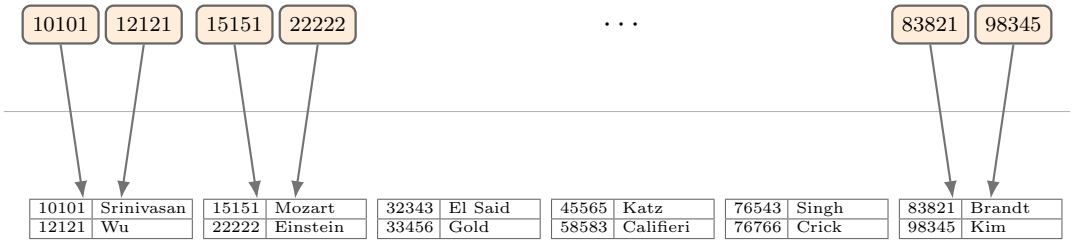


nearby index entries lead to nearby data — matching records are scattered across the file

This difference matters because:

- a clustered index keeps matching records on nearby pages, so range queries are efficient,
- an unclustered index may jump to many unrelated pages, so large result sets become expensive,
- a file can have many unclustered indexes, but only one clustered physical order at a time.

Some textbooks use *primary* for clustered and *secondary* for unclustered indexes. That vocabulary is not universal across DBMSs, so the physical term *clustered* is safer.



clustered index on `ID`: nearby index entries point to nearby data pages

In the example above, the data file is ordered on `ID`. Nearby index entries lead to nearby data pages, which is exactly the clustered case.

3.6.2 Dense and sparse indexes

Dense index. Has an index entry for every search-key value that appears in the data file.

Sparse index. Has entries for only some search-key values, usually one entry per data page or page range.

A sparse index uses less space and needs fewer updates, but it works well only when the data file is ordered on the search key. After the sparse index points to the right page, the DBMS still searches within that page.

Range scans: why clustering matters

For range predicates, the difference between clustered and unclustered indexes becomes much larger:

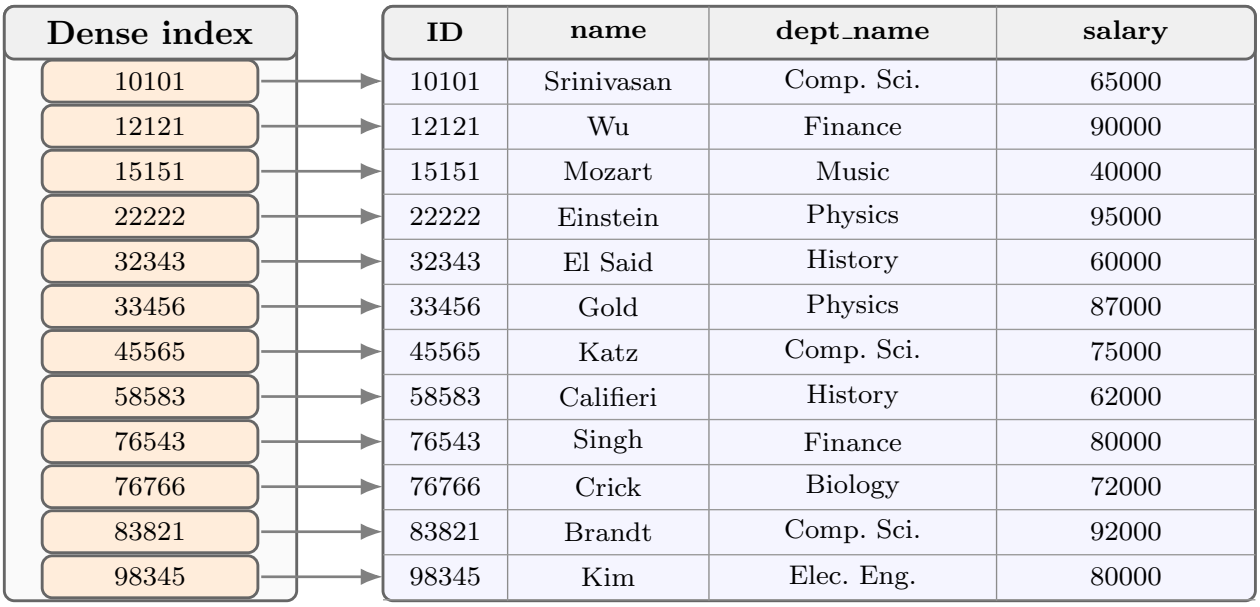
- with a **clustered** index, cost is roughly the number of **data pages** containing matching records,
- with an **unclustered** index, cost is closer to the number of **matching index entries**, because each match may lead to a different data page.

So the trade-off is:

- **clustered advantage**: very efficient for range queries and ordered scans,
- **clustered drawback**: expensive to maintain, because updates disturb the physical order,
- **unclustered advantage**: easy to add as an extra access path without reorganizing the data file,
- **unclustered drawback**: large result sets can cause many random I/Os.

Dense index on a key attribute

If the search key is a key, each value identifies one record. Then a dense index has one entry per record, and each entry can point directly to one RID.



ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dense index on a key: one index entry per record

Dense index on a non-key attribute

If the file is sorted on a non-key attribute, equal values appear together. A dense index can then keep one entry for each **distinct** search-key value and point to the first record of that group.

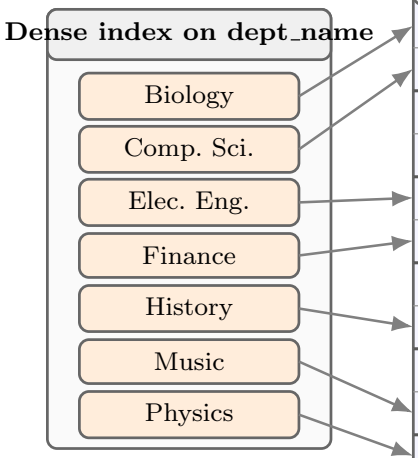
Sparse index files

Sparse indexes are most useful when the data file is already ordered on the search key. A common design keeps one anchor entry per data page.

To locate a record with search-key value K:

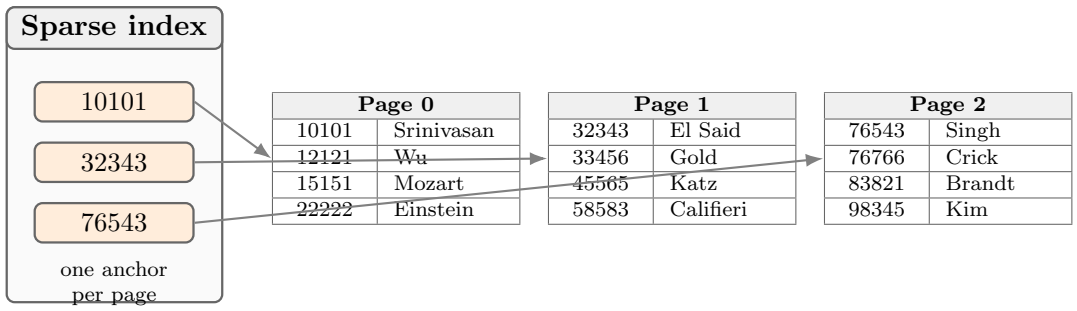
- find the index entry with the largest key value $\leq K$,
- follow that pointer to the corresponding data page,
- scan forward in the file until K is found or passed.

Because this method relies on sequential order in the data file, a sparse index is normally clustered.



ID	name	dept_name	salary
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

one entry per distinct search-key value, pointing to the first matching record



to search for K, jump to the page with the largest anchor key $\leq K$ and scan forward

3.6.3 Key and non-key search keys

If the search key is a key, each value identifies at most one record, so an index entry can point to one RID directly. If the search key is not a key, duplicates are possible, so one entry may need to point to a list or bucket of RIDs.

Physical order also matters. If the file is ordered on the indexing field, clustered and sparse organizations become possible. If it is not, the index is typically dense and unclustered.

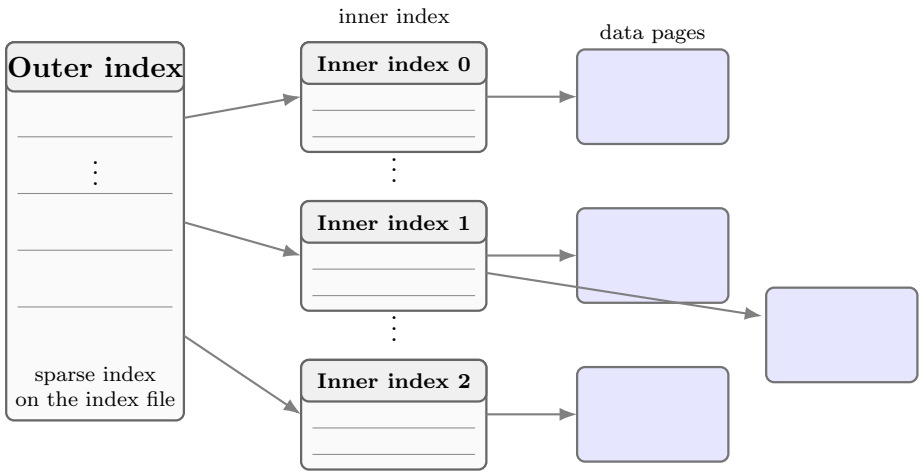
3.6.4 Multilevel indexes

If an index itself spans many pages, finding the right index page can become expensive.

Multilevel index. Treat the index file as a sorted file and build a sparse index on top of it. Repeat this until the top level is small enough to search very cheaply.

In a two-level design:

- the **inner index** is the ordinary index on the data file,
- the **outer index** is a sparse index on the pages of the inner index.



if one index is too large, build a sparse index on top of it

Search descends level by level until it reaches the target data page. If the outer index also becomes too large, another level can be added. The downside is that insertions and deletions may force updates at several levels.

3.6.5 Secondary indexes

Secondary index. An index whose search key does not define the physical order of the data file. In practice, this means the index is unclustered.

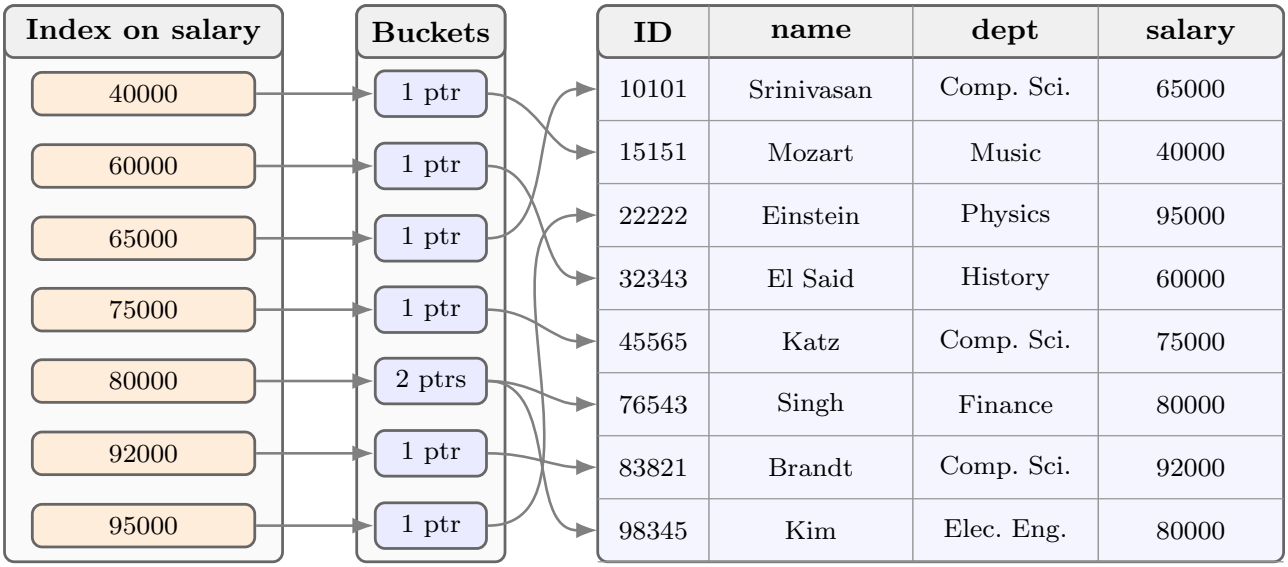
For a non-key search key, a secondary index usually works as follows:

- one index entry is stored for each search-key value,
- the index entry points to a bucket (or list) of RIDs,
- the bucket contains pointers to all matching records.

Secondary indexes on unsorted columns must be dense. Otherwise some matching records would not be reachable from the index.

3.7 B+-tree index files

Sequential dense and sparse indexes explain the core ideas clearly, but real DBMSs usually prefer B^+-trees because they stay balanced under updates.

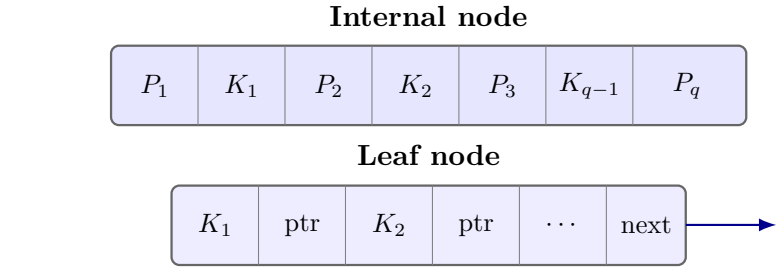


secondary index on an unsorted field: one entry per value, pointing to a bucket of record pointers

B+-tree. A balanced search tree whose nodes are page-sized blocks. Internal nodes store separator keys and child pointers; leaf nodes store sorted search-key entries and are linked left-to-right for efficient range scans.

For a tree of order n:

- all paths from the root to a leaf have the same length,
- each non-root internal node has between $\lceil n/2 \rceil$ and n children,
- each leaf has between $\lceil (n-1)/2 \rceil$ and $n-1$ key values,
- if the root is not a leaf, it has at least 2 children,
- if the root is also a leaf, it may contain between 0 and $n-1$ values.

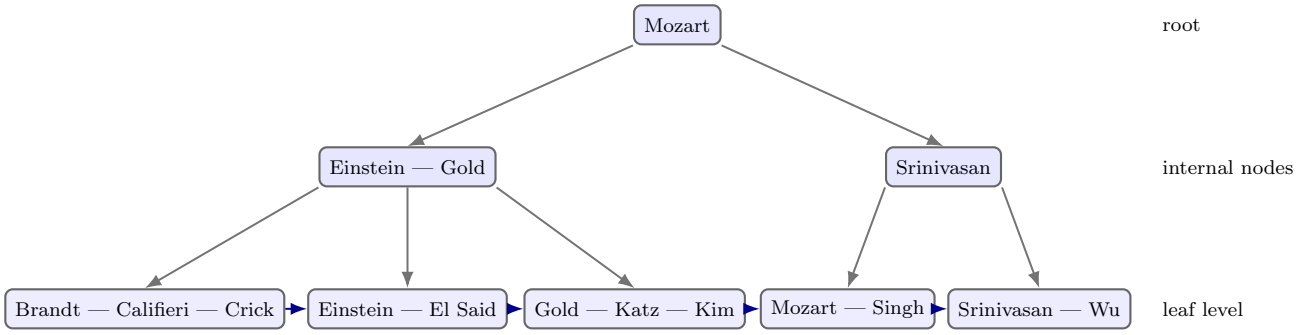


internal nodes route the search; leaves store actual entries and are linked for range scans

If an internal node has child pointers $P_1, \ldots, P_q$, it stores separator keys $K_1, \ldots, K_{q-1}$. Each key K_i separates the subtree reached through P_i from the subtree reached through P_{i+1}.

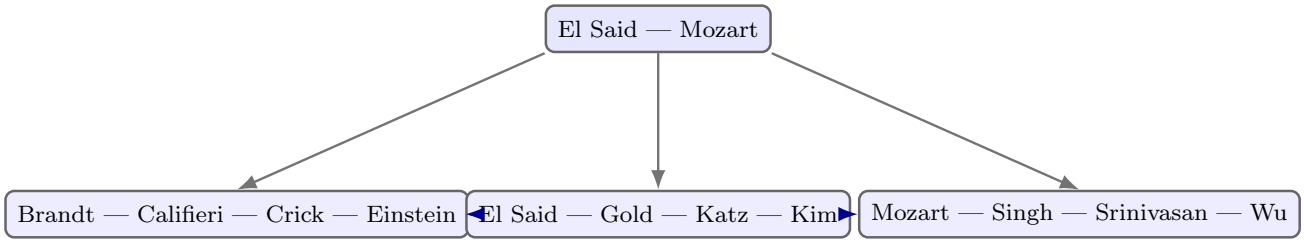
Example: order 4

With a small order, each node has limited fan-out, so the tree becomes wider and deeper more quickly. The leaf level is linked in sorted order, so once the first matching leaf is found, a range scan can continue sequentially.



order 4: small fan-out, more levels

Example: order 6



order 6: larger fan-out keeps the tree shallower

A larger order means more keys and child pointers fit in each node, so the same data can be indexed with fewer levels. This is why disk-based trees aim for high fan-out.

3.7.1 Revisiting clustered and unclustered terminology

To keep the vocabulary straight:

- **clustered / primary index**: the file is physically ordered on the indexing field; index entries may point to blocks or to individual records; the index can be sparse or dense,
- **unclustered / secondary index**: the file is not physically ordered on the indexing field; index entries point to records or to buckets of record pointers; the index must be dense.

So the core question is always physical order: does the data file itself follow the search-key order, or not?

3.8 Hash-based versus tree-based indexes

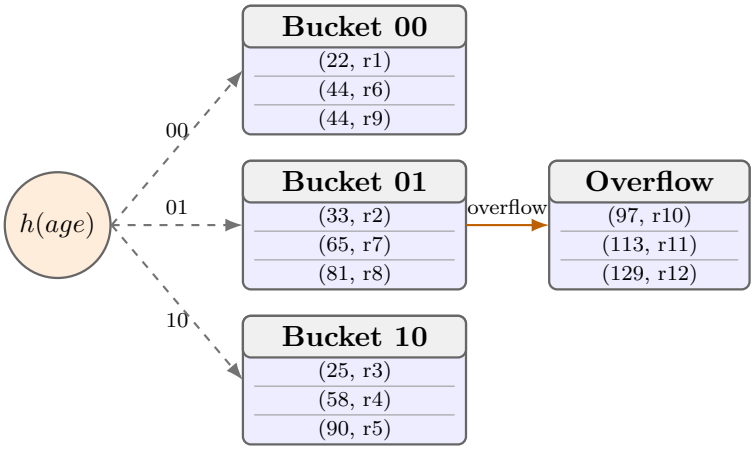
The two main index families solve different problems. Hash indexes optimize exact lookup. Tree indexes optimize ordered access.

3.8.1 Hash-based indexes

Hash-based index. Uses a hash function h on the search-key value to choose a bucket. Each bucket has one primary page and may have overflow pages if collisions accumulate.

Hash indexes are best for equality predicates:

- to evaluate `age = 44`, the DBMS computes $h(44)$ and reads the corresponding bucket,
- duplicate values naturally land in the same bucket,
- inserts and deletes usually affect one bucket only,
- the entries are **not** kept in sorted order.



hash index: equality lookup computes one bucket; collisions create overflow pages

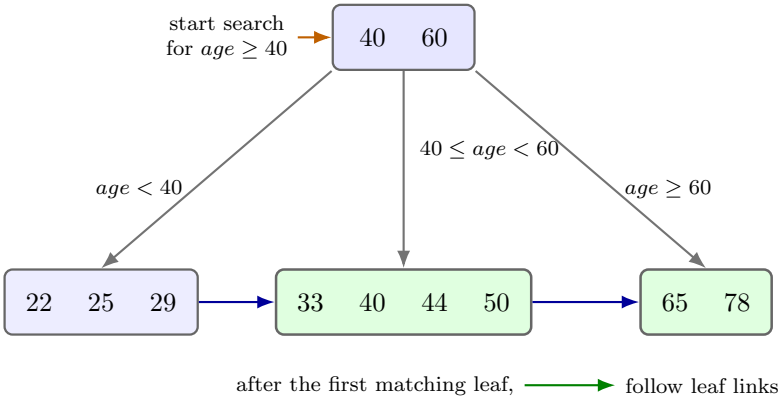
Because order is lost, hash indexes are poor for range predicates such as `age > 40` or `3.0 <= gpa <= 3.5`. The DBMS would need to inspect many buckets, which is close to scanning the whole file.

3.8.2 Tree-based indexes

Tree-based index. Keeps entries sorted by search-key value. In practice, this is usually implemented as a B^+-tree.

Tree indexes are good for range predicates:

- leaf entries are sorted by the search key,
- all root-to-leaf paths have the same length, so access remains balanced,
- after the first matching leaf is found, the DBMS follows the leaf links to continue the scan.



A tree index is more flexible: the same structure supports equality search, ordered scans, prefix lookups on composite keys, and range scans.

3.8.3 Indexing decisions follow queries

```
SELECT E.dno
FROM Employees E
WHERE E.age > 40
```

This query already suggests the right index family:

- `age > 40` is a range predicate, so a tree index on `age` is the natural choice,
- a hash index on `age` would not help, because it only groups equal values,
- if many qualifying tuples are expected, a **clustered** tree index is much better than an unclustered one,
- if the predicate matches a large fraction of the relation, a sequential scan may still be cheaper than using the index.

Selective predicate. Matches a small fraction of the relation. Indexes help most in this case; if most records qualify, scanning is often cheaper.

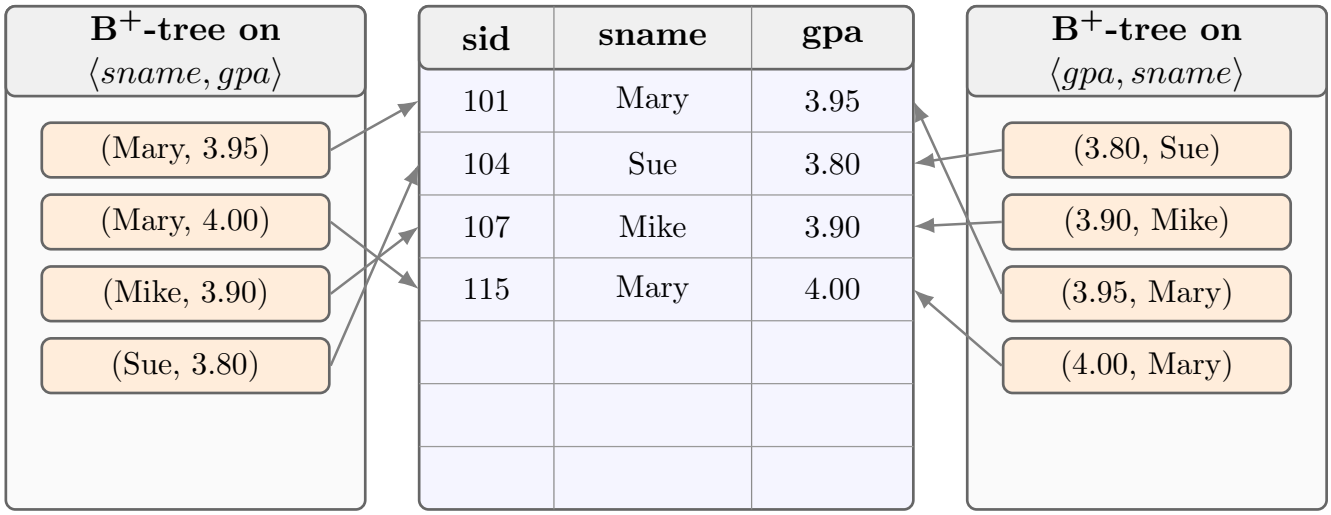
3.9 Choosing a search key

```
SELECT sid
FROM Student
WHERE sname = 'Mary'
  AND gpa > 3.9
```

A good search key depends on which part of the predicate does the filtering:

- **index on** `sname`: useful because `sname = 'Mary'` is an equality condition. A hash index is a reasonable choice if this kind of lookup dominates.
- **index on** `gpa`: useful because `gpa > 3.9` is a range condition. This should be a tree index.
- **index on** $\langle$`sname`, `gpa`$\rangle$: often the best single index for this query. A tree index first narrows to `sname = 'Mary'`, then scans only the qualifying GPA values inside that group.

Lexicographic order. In a composite tree index on $\langle A, B\rangle$, entries are sorted first by A and, among equal A values, by B.



same attributes, different order: the useful predicates change with the lexicographic order

Field order matters. An index on $\langle$`sname`, `gpa`$\rangle$ is much better for the query above than an index on $\langle$`gpa`, `sname`$\rangle$.

3.9.1 Composite search keys

Composite search key. An index whose search key is a tuple of attributes rather than one single attribute.

For composite keys, the shape of the predicate matters:

- **full equality**: with `age = 12 AND sal = 75`, both a hash index and a tree index on $\langle$`age`, `sal`$\rangle$ can work.
- **prefix equality**: with `age = 12`, a tree index on $\langle$`age`, `sal`$\rangle$ still works because all entries with the same age are adjacent. A hash index on the full pair does not.
- **prefix equality + range**: with `age = 12 AND sal > 20`, a tree index on $\langle$`age`, `sal`$\rangle$ is ideal. An index on $\langle$`sal`, `age`$\rangle$ is much less useful for that query.

Leftmost-prefix rule. A composite tree index on $\langle A, B, C\rangle$ can be used efficiently only when the query constrains a prefix of the key: A, or A and B, or A, B, and C.

3.9.2 Composite keys and index-only evaluation

```
SELECT AVG(E.sal)
FROM Employees E
WHERE E.age = 25
  AND E.sal BETWEEN 3000 AND 5000
```

For this query:

- the right structure is a tree index, not a hash index, because `BETWEEN` is a range predicate,
- the better order is $\langle$`age`, `sal`$\rangle$, because the search starts with one exact age and then scans only the salary interval inside that age group,
- an index on $\langle$`sal`, `age`$\rangle$ would first scan the salary interval for *all* ages and then filter `age = 25`,
- the query needs only `sal`, which already appears in the composite key.

Index-only evaluation. If every attribute needed by the query is already present in the index entries, the DBMS can answer the query from the index alone, without fetching the full data records.

This is also called a **covering index**. It is especially attractive for aggregates such as `AVG`, `COUNT`, `MIN`, and `MAX`.

3.10 System catalogs

System catalog. The DBMS's metadata repository. It stores information about relations, attributes, indexes, views, constraints, statistics, and access control.

The catalog typically stores:

- **for each relation**: relation name, backing file, file organization, attribute names and types, and integrity constraints,
- **for each index**: index name, structure (B^+-tree, hash, ...), search-key fields, and clustering status,
- **for each view**: view name and definition,
- **for the optimizer and system**: statistics such as relation cardinality, page count, average row length, plus authorization and other metadata.

Catalogs are themselves stored as relations, so the DBMS can query them like ordinary tables.

3.10.1 Example: Oracle system catalogs

Oracle exposes catalog information through views such as `USER_TABLES` and `USER_TAB_COLUMNS`.

```
SELECT *
FROM student
```

student

SID	NAME	AGE
A1234	John	23
B1	Xavier	24
A21341	Scott	21
C12948291	Benjamin	25
1948	Ben	29

```
SELECT table_name, num_rows, blocks, avg_row_len
FROM user_tables
```

TABLE_NAME	NUM_ROWS	BLOCKS	AVG_ROW_LEN
STUDENT	5	5	20

```
SELECT table_name, column_name, data_type, data_length
FROM user_tab_columns
WHERE table_name = 'STUDENT'
```

TABLE_NAME	COLUMN_NAME	DATA_TYPE	DATA_LENGTH
STUDENT	SID	CHAR	10
STUDENT	NAME	VARCHAR2	32
STUDENT	AGE	NUMBER	22

These catalog views are what the optimizer and storage manager rely on: they describe the schema, give rough size estimates, and tell the system which access paths already exist.

Chapter 4

L4 - The Storage Layer & Buffer Management

4.1 The storage hierarchy

A DBMS does not operate on one uniform memory. Data moves across a hierarchy of storage levels with very different latency, capacity, and cost.

- **CPU registers, caches, DRAM**: volatile, byte-addressable, and very fast.
- **SSD, HDD, network storage**: persistent, block-addressable, much larger, and much slower.
- Going *up* the hierarchy gives lower latency but less capacity and a higher cost per byte; going *down* gives the opposite.

Main memory holds the **working set**: the data currently being processed. Persistent storage holds the full database. Modern hardware adds intermediate levels such as **persistent memory** and **fast network storage**. Still, the DBMS must move pages carefully between storage and RAM.

Typical latency numbers

Operation	Latency	If 1 ns were 1 s
L1 cache reference	1 ns	1 s
L2 cache reference	4 ns	4 s
DRAM access	100 ns	100 s
SSD access	16,000 ns	4.4 h
HDD access	2,000,000 ns	3.3 weeks
Network storage	$\sim$ 50,000,000 ns	1.5 years
Tape archive	1,000,000,000 ns	31.7 years

The exact values vary by hardware, but the orders of magnitude do not: one extra I/O can cost more than millions of CPU instructions.

4.2 Why DBMSs still care about disks

A DBMS usually keeps only the actively used data in RAM and stores the database itself on secondary storage. This creates two expensive operations:

- **read**: transfer a page from disk to main memory,
- **write**: transfer a modified page from main memory back to disk.

Relative to in-memory operations, both are expensive, so the DBMS must plan I/O carefully.

4.2.1 Why not keep everything in main memory?

- **Cost**: large production databases can reach the TB or PB scale, so all-DRAM storage is expensive.
- **Volatility**: conventional DRAM loses data on power failure, but databases need persistence across runs.
- **Main-memory DBMSs**: useful for smaller or latency-critical workloads; larger memories and non-volatile devices make them increasingly practical.

For general-purpose systems, disks remain the default place to store data.

4.2.2 Pages and storage devices

Data is stored and retrieved in units called **pages** (or **disk blocks**), not byte by byte.

Page (disk block). The basic unit of transfer between secondary storage and main memory. DBMS I/O is organized around whole pages.

Access cost depends on both the device and the access pattern:

- **magnetic disks (HDDs)**: very large capacity and low cost per byte, but slow random access; good streaming throughput.
- **flash storage (SSDs)**: lower latency and strong random-read performance, but still far slower than DRAM and more constrained on writes.
- **layout matters**: unlike RAM, page access time depends heavily on where a page is placed and whether access is sequential or random.

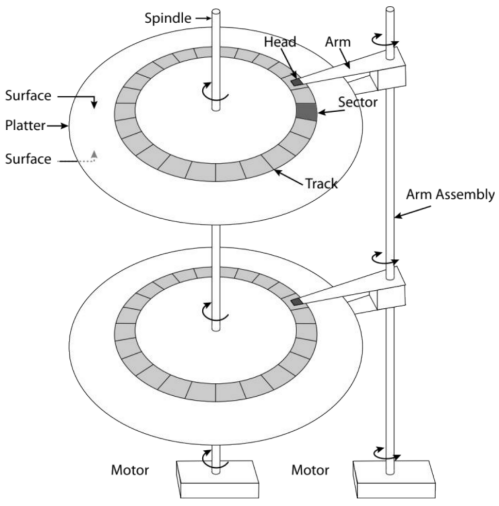
4.3 Magnetic disk organization

A magnetic disk stores bits on rotating platters coated with magnetic material. Each disk surface is organized geometrically.

Track. A concentric circle on one disk surface.

Sector. A slice of a track and the smallest hardware-addressable unit on the disk.

Cylinder. The set of tracks at the same radius across all disk surfaces.



Important details:

- outer tracks are longer, so they can store more sectors and offer higher transfer bandwidth than inner tracks,
- disks are divided into **zones**: all tracks in one zone have the same number of sectors per track,
- the hardware exposes a sector-addressable space; common sector sizes are 512 B or 4096 B,
- a DBMS page size is a fixed multiple of the sector size,
- the main operations are **read** and **write**.

4.3.1 Cost of accessing a page

For a magnetic disk, page access time is well approximated by

$$\text{access time} = \text{seek time} + \text{rotational delay} + \text{transfer time}.$$

Seek time. Time to move the disk arm so the head reaches the correct track. Typically, 5–15 ms.

Rotational delay. Waiting time until the desired sector rotates under the read/write head. At 7200 RPM, one rotation takes about 8.3 ms, so the average delay is about half of that.

Transfer time. Time to actually move the bytes once the head is already in position. For one page, this is usually much smaller than seek and rotation.

This is why **sequential access** is much faster than random access: once the head is already near the right place, the DBMS can read neighboring blocks with little extra positioning cost.

4.3.2 Arranging pages on disk

To minimize latency, blocks of the same file should be laid out in a sensible physical order. A good notion of the **next** block is:

- first, a block on the same track,
- then, a block on the same cylinder,
- then, a block on an adjacent cylinder.

Pre-fetching. Reading nearby pages before they are explicitly requested, because sequential layout makes them cheap to fetch once the disk is already positioned.

Physical placement matters a lot: a disk block has several useful physical neighbors, not just one logical successor.

4.3.3 A simple disk-performance model

Assume a magnetic disk with:

- average seek time: 5 ms,
- spindle speed: 7200 RPM $\Rightarrow$ one rotation in $\frac{60{,}000}{7200} \approx 8.33$ ms, so average rotational delay ≈ 4.17 ms,
- transfer rate: 50 MB/s,
- page size: 4096 B $\Rightarrow$ transfer time $= \frac{4096}{50\times10^6}$ s ≈ 0.082 ms.

Example: 4 KB reads on a 7200 RPM HDD

Access pattern	Time	Throughput
Random block anywhere on disk	$5+4+0.082 = 9.082$ ms	≈ 451 KB/s
Random block in the same cylinder	$4 + 0.082 = 4.082$ ms	≈ 1.03 MB/s
Next block on the same track (sequential)	0.082 ms	≈ 50 MB/s

The gap is enormous: good physical layout and sequential scans can improve effective bandwidth by two orders of magnitude. Page placement and clustering matter. Buffer management builds on exactly this gap.

4.3.4 Rules of thumb

Two heuristics explain most storage-level design decisions:

- main-memory access is roughly $\sim 1000\times$ faster than disk I/O,
- on magnetic disks, sequential I/O is roughly $\sim 10\times$ faster than random I/O.

Thus, DBMSs try to keep hot data in memory and lay out pages sequentially on secondary storage.

4.4 Flash storage

Flash memory has existed for decades and evolved from USB keys and mobile-device storage into consumer and enterprise **SSDs**. In DBMSs, it can serve either as **main storage** or as an **accelerator** such as a cache or logging device.

4.4.1 Why flash changed storage

Flash disks are one of the main modern solutions for fast storage:

- they offer orders-of-magnitude higher I/O performance than HDDs,
- they expose the same block-oriented API as disks, so DBMSs can still work with pages,
- there is no mechanical seek, so **random reads** are close in cost to **sequential reads**,
- unlike HDDs, access time is not determined by physical page location on the device.

Flash page and flash block. Reads and writes operate at **page** granularity, while erases operate at **block** granularity. A flash block contains many pages.

4.4.2 Internal organization

An SSD is a small parallel system rather than a single chip:

- a controller runs firmware and exports a disk-like block interface,
- flash packages are connected through **channels**,
- each package contains several **dies** (chips),
- each die contains **planes**, **blocks**, and **pages**.

This is why flash access time depends on controller design, internal parallelism, firmware, and package bandwidth, not on a moving disk head. In typical devices, a block may contain dozens of 4 KB pages.

4.4.3 Operations, density, and lifetime

Flash supports three basic operations:

- **read** and **write** at page granularity,
- **write** can only program cells from $1 \to 0$,
- **erase** resets cells from $0 \to 1$, but only at block granularity.

Common flash families

Type	Bits/cell	Typical trade-off
SLC	1	fastest, longest lifetime, highest cost
MLC	2	more capacity, lower endurance
TLC	3	cheaper per bit, lower endurance and slower writes
QLC	4	highest density, lowest endurance; best for read-heavy use

As density rises from **SLC** to **QLC**, cost per bit falls, but endurance and write performance usually decline. Device lifetime often ranges roughly from 10^5 down to 10^3 **program/erase (P/E) cycles**.

Wear leveling. Firmware techniques that spread writes and erases across blocks so that no small region of flash wears out too early.

4.4.4 Flash Translation Layer (FTL)

Flash Translation Layer (FTL). The firmware layer that gives SSDs an HDD-like interface. It remaps logical pages to physical flash locations, performs garbage collection, applies wear leveling, and tunes both performance and device lifetime.

The FTL is necessary because flash cannot overwrite pages in place the way magnetic disks can.

4.4.5 Flash versus HDDs

In practice, the two technologies serve different goals:

- **HDDs**: best when the main goal is **large, inexpensive capacity**.
- **Flash**: best when the workload needs **low latency**, **high IOPS**, and especially **efficient random reads**.

4.5 Storage requirements and RAID

Storage systems should ideally be **fast**, **reliable**, and **affordable**. A single disk rarely optimizes all three at once.

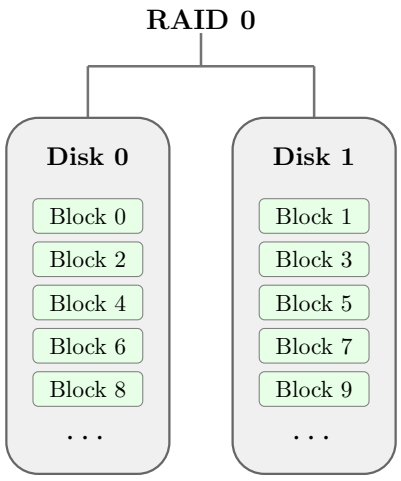
RAID. Redundant **A**rray of **I**nexpensive **D**isks. RAID builds one logical disk from several physical disks to improve throughput, reliability, or both.

RAID combines two main ideas:

- **striping**: spread data across disks for higher throughput,
- **redundancy**: duplicate or protect data so the array can survive failures.

4.5.1 RAID 0: striping

- file blocks are striped across disks,
- there is **no redundancy**, so RAID 0 provides **no fault tolerance**,
- throughput is excellent because the array can use the cumulative bandwidth of all disks,
- usable capacity is the sum of the capacities of all disks.

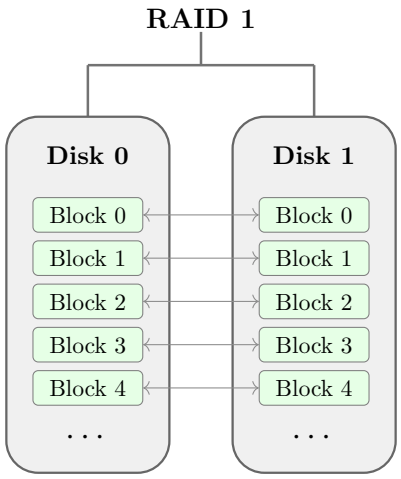


The downside is reliability: with N independent disks, the mean time to failure of the striped array is roughly $\frac{1}{N}$ of the mean time to failure of one disk, because *any* disk failure breaks the array.

4.5.2 RAID 1: mirroring

- every block is duplicated on another disk,
- RAID 1 handles **disk loss** well, but it does **not** protect against logical corruption or bad writes mirrored to both copies,
- usable capacity is only that of one disk per mirror pair,
- reads can be parallelized, but each write must update all mirrored copies.

RAID 1 is simple and reliable, but expensive: half of the raw capacity is used for redundancy.



4.5.3 RAID 5: striping with rotating parity

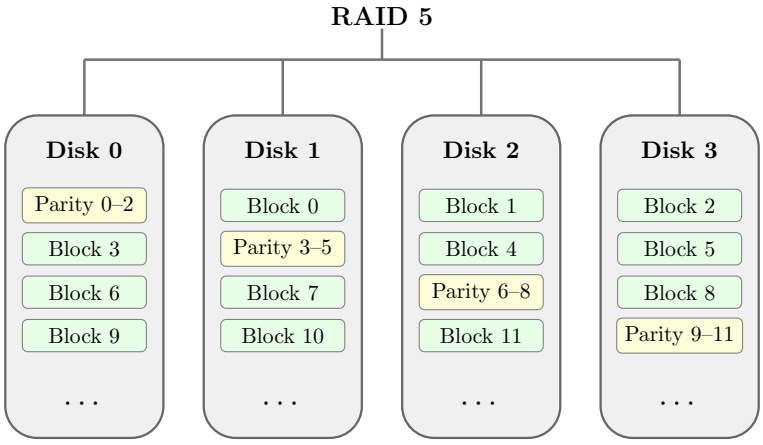
Parity. An extra block computed as the bitwise XOR of several data blocks. If one block is missing, it can be reconstructed from the parity block and the surviving data blocks.

For one stripe group,

$$P = S_0 \oplus S_1 \oplus \cdots \oplus S_{N-2}.$$

- data is striped across disks, but parity blocks rotate from disk to disk,
- the array can survive the loss of **one** disk,
- usable capacity is the equivalent of $N - 1$ disks out of N,
- reads are fast, but writes are more complex because parity must be updated.

RAID 5 is a common compromise: it is more affordable than mirroring, more reliable than pure striping, and widely used in servers and datacenters.



4.6 Disk space management

Disk space management. The lowest storage-oriented layer of the DBMS. It manages free space on disk. It also exports page-level operations such as allocate, deallocate, read, and write.

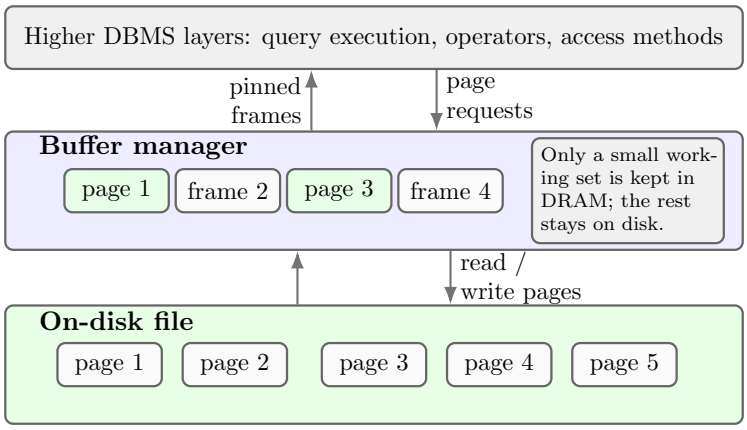
Higher layers should not have to manage physical layout directly. They request pages, while this layer decides where pages live on disk and how free space is tracked.

The best case is when a request for a **sequence of pages** is satisfied by pages stored sequentially on disk. Then the DBMS gets the full benefit of fast sequential I/O without exposing storage details to upper layers.

In the DBMS stack, **buffer management** sits directly above disk space management. Higher levels ask for pages in memory, and the buffer manager turns those requests into page I/O.

4.7 Buffer management

A DBMS can only process data that is currently in DRAM. The buffer manager hides the fact that most pages still live on disk, much like a hardware cache hides the gap between CPU and main memory.



4.7.1 Why DBMSs do not rely only on the OS

The operating system already provides a file system and a block cache, but DBMSs still want to manage pages their own way:

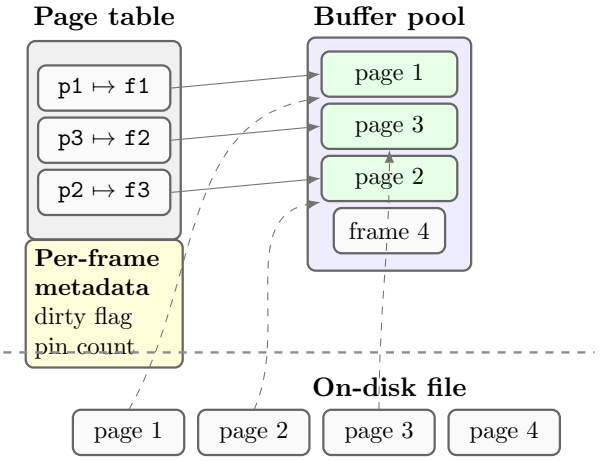
- **custom prefetching**: many database scans are predictable, so the DBMS can fetch pages ahead of time,
- **replacement control**: the DBMS wants to choose which pages stay in memory instead of accepting a generic OS policy,
- **flushing control**: recovery protocols such as **WAL** may require log records to reach disk before dirty data pages are written,
- **scheduling control**: OS scheduling can interfere with DBMS locking and create **convoy effects**, where one blocked worker delays many others.

4.7.2 Buffer pool, frames, and metadata

Buffer pool. A region of DRAM organized as an array of fixed-size slots used to cache database pages.

Frame. One slot in the buffer pool. When the DBMS fetches a page, an exact copy of that page is placed into one frame.

Page table. An in-memory mapping from page IDs to the frames that currently hold them. It exists only in memory and is not stored on disk.



For each cached page, the buffer manager also stores metadata:

- **dirty flag**: whether the in-memory copy was modified and must be written back before eviction,
- **pin count** (reference count): how many DBMS components are currently using the page,
- **replacement metadata**: for example timestamps, access counters, or reference bits.

Dirty pages are often kept in memory for a while instead of being written back immediately. This avoids turning every update into a synchronous disk stall.

4.7.3 Handling a page request

Pinned page. A page whose pin count is greater than zero because some operator or transaction is actively using it. Pinned pages are not valid eviction candidates.

When higher layers request a page:

- on a **hit**, the page is already in the buffer pool, so the DBMS returns its frame and increments the pin count,
- on a **miss**, the DBMS must first find a frame:
  - if an empty frame exists, use it,
  - otherwise choose a victim according to the replacement policy,
  - only pages with pin count $= 0$ are candidates.
- if the chosen victim is **dirty**, write it back to disk first,
- read the requested page into the chosen frame,
- pin the page and return its address to the caller.

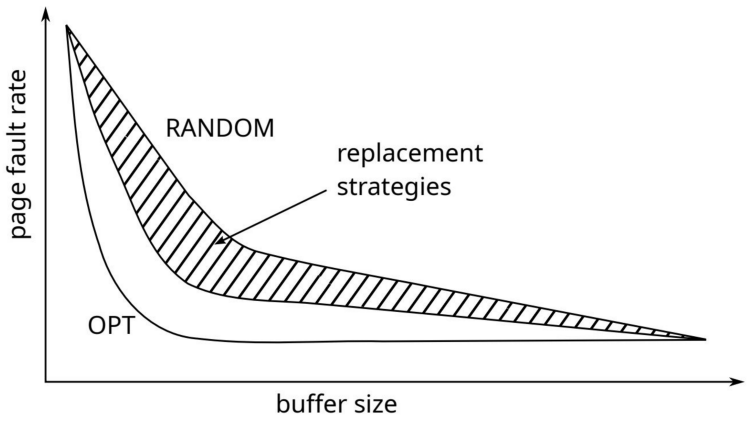
When the caller finishes, it must **unpin** the page and indicate whether the page was modified. A page may be pinned several times, so it becomes evictable only when its pin count returns to 0.

If the request pattern is predictable, for example during a long sequential scan, the buffer manager can **pre-fetch** several pages at once. Replacement can also trigger extra I/O for recovery, because **write-ahead logging** may force older log records to disk before a dirty data page is flushed.

4.7.4 Replacement policies

When the buffer pool is full, some cached page must be replaced:

- a **clean** page can be discarded immediately,
- a **dirty** page must first be written back,
- the policy should balance correctness, hit rate, speed, and metadata overhead.



Common policies include **FIFO**, **LRU**, **MFU**, **Clock**, **LRU-K**, and **2Q**. In practice, DBMSs prefer policies that are both effective and cheap to maintain online.

FIFO (first-in-first-out)

- keep frames in queue order,
- insert newly loaded pages at the tail and evict from the head,
- very simple, but it ignores locality: an old page may still be hot and yet be removed first.

FIFO is easy to implement, but it does **not** reliably retain frequently reused pages.

LRU (least recently used)

- evict the unpinned page whose last access is the oldest,
- conceptually this means keeping a timestamp per page,
- a practical implementation keeps frames in recency order, often with a doubly-linked list,
- frequently reused pages move toward the back and tend to stay cached.

LRU is one of the most widely used policies because it exploits temporal locality well.

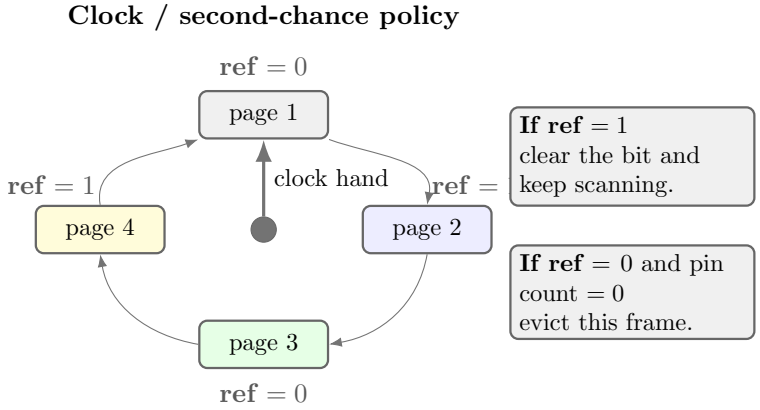
Clock (second chance)

Clock policy. An approximation of LRU that stores one **reference bit** per frame instead of a full access timestamp.

The policy organizes frames conceptually in a circle with a moving **clock hand**. Each access sets the page's reference bit to 1. When a victim is needed:

- if the current frame is pinned, skip it,
- if its reference bit is 1, clear the bit and advance,
- if its reference bit is 0, evict it.

A hit does not move the page to a different frame; it only turns the reference bit back on. This makes Clock cheaper than exact LRU while keeping much of the same intuition.



MFU

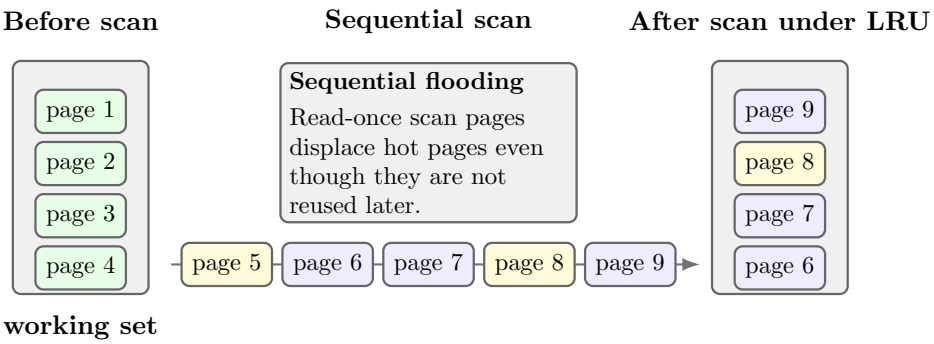
- **MFU** (most frequently used) uses access frequency rather than recency,
- it is less common in practice, but it shows that reuse history can be richer than "last access only".

Sequential flooding

Scan resistance. The ability of a replacement policy to avoid polluting the buffer pool with pages that are touched once during a large scan and then never reused.

Both **LRU** and **Clock** can suffer from **sequential flooding**. A large sequential scan may push useful pages out of memory even though the scanned pages are not part of the long-term working set.

- the query reads pages $p_5, p_6, \ldots$ one by one,
- each scanned page causes a miss and briefly enters the pool,
- if #frames < #pages in the file, then almost every request triggers an I/O,
- the buffer pool ends up filled with read-once pages, while previously hot pages have been evicted.



This is why modern policies try to distinguish **reuse** from a one-time access.

LRU-K

LRU-K. An extension of LRU that tracks the last K references to each page and uses this history to estimate whether the page is likely to be reused.

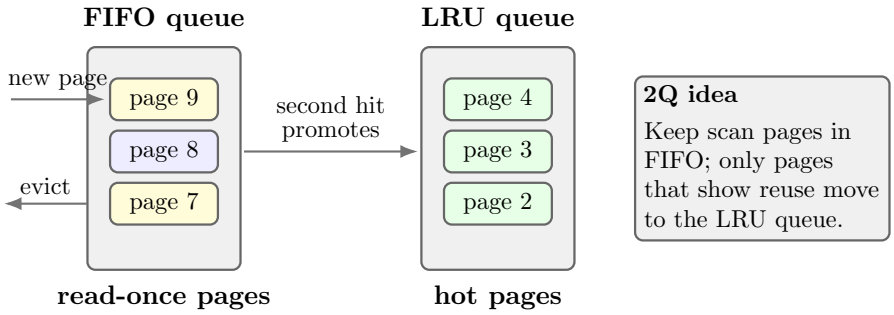
- pages with fewer than K recorded accesses are **cold**,
- pages with at least K accesses are **warm**,
- cold pages are evicted before warm pages,
- if all candidates are warm, compare the K-th most recent access and evict the page whose K-th access is the oldest,
- when $K = 1$, LRU-K reduces to ordinary LRU.

Because one-off scan pages usually remain **cold**, LRU-K is much more resistant to sequential flooding.

2Q

2Q. A scan-resistant policy that keeps two queues: a FIFO queue for recently seen pages and an LRU queue for pages that proved they are reused.

- new pages first enter the **FIFO** queue,
- if a page is referenced again while it is still known to the system, it is promoted to the **LRU** queue,
- evictions prefer the FIFO queue, so read-once scan pages are discarded there,
- hot pages stay in the LRU queue and are protected from scan pollution.



In short: **read-once pages stay in FIFO, reused pages graduate to LRU**. This makes 2Q simple, practical, and much more scan resistant than plain LRU or Clock.

Chapter 5

L5 - Access Methods: Hashing and B$^+$-Trees

5.1 Where access methods fit

The DBMS execution engine reads and writes **pages**. To avoid scanning every page for every query, the DBMS uses **access methods**: physical data structures that help it find the right page or record quickly.

At this layer, two large families dominate:

- **tree-based structures**: ordered access paths,
- **hash-based structures**: unordered access paths.

This difference is the key idea of the lecture:

- if the workload needs mainly **equality search**, hashing is often attractive,
- if the workload needs **equality and range search**, ordered trees are the standard choice.

5.2 How an index points to table data

Data entry k^*. The information stored in the index for search-key value k. It tells the DBMS how to reach the matching table record or records.

There are three classic representations for a data entry:

Three alternatives for index data entries

Alt.	Form of entry	Meaning
1	actual data record with key k	The index entry is the record itself. The index and the data file are physically the same structure.
2	$\langle k, \text{RID} \rangle$	The index stores the key and one pointer to one matching record.
3	$\langle k, \text{list of RIDs} \rangle$	The index stores one key and a list of pointers to all matching records.

The important point is that this choice is **orthogonal** to the indexing technique:

- a **hash index** can use Alternative 1, 2, or 3,
- a **tree index** can also use Alternative 1, 2, or 3.

5.2.1 Reading the three alternatives

- **Alternative 1**: the leaf level stores the full records. Very fast because no extra jump from index entry to data record is needed.
- **Alternative 2**: one pair $\langle k, \text{RID} \rangle$ per matching record. Simple, works well even when the search key is not unique.
- **Alternative 3**: one key with a full RID list. Saves space when many records share the same key value.

5.2.2 Why Alternatives 2 and 3 are often preferred

Suppose an employee file is itself hashed on `age`. Then the buckets store the **actual employee records**: this is Alternative 1.

Now suppose we also want an index on `salary`. We usually do *not* want to duplicate the full records inside a second structure. Instead, we create another access path that stores either:

- $\langle$`salary`, RID$\rangle$ pairs, or
- $\langle$`salary`, list of RIDs$\rangle$ entries.

This leads to the standard trade-offs:

- Alternatives 2 and 3 are easier to maintain than Alternative 1.
- If a file has several indexes, only **one** of them can use Alternative 1, because the data records can only be physically organized one way at a time.
- Alternative 3 is usually more compact than Alternative 2 when many duplicates exist.
- The downside of Alternative 3 is that entries become **variable-sized**, even when the search key has fixed length. If one key has a very long RID list, that entry may span several pages.

5.3 Hash indexes versus tree indexes

Hashing and trees solve different problems.

Hash-based index. Uses a hash function $h(k)$ to send a search-key value to one bucket. Because the buckets are not ordered by key value, hashing is best for equality search.

Tree-based index. Keeps entries in sorted search-key order. In practice, the standard disk-oriented tree structure is the B^+-tree.

Hash indexes and tree indexes

Structure	Best at	Main limitation
Hash index	equality predicates such as `age = 44`	no natural order, so range predicates are poor
B^+-tree index	equality, range, ordered scans, sequential access	usually a little more overhead for pure equality lookups

5.3.1 Why range search needs order

Consider the query:

```
SELECT *
FROM Student
WHERE gpa > 3.0
```

If the data file were physically sorted on `gpa`, the DBMS could:

- do a binary search to find the first matching student,
- then scan forward to collect the rest.

The problem is that keeping the *whole data file* sorted is expensive: insertions and deletions constantly disturb the order.

The simple idea is therefore:

- keep a smaller **index file** in sorted order,
- search the index first,
- then use it to reach the data pages.

This is the basic intuition behind the B^+-tree: keep the *index* ordered, not the whole relation.

5.4 B$^+$-trees

B$^+$-tree. A self-balancing, ordered tree structure optimized for page-based storage. Search, insertion, deletion, and sequential access all work efficiently, typically in $O(\log_F N)$, where F is the fanout and N is the number of leaf nodes.

Fanout. The maximum number of child pointers an internal node can hold. Large fanout is what makes disk-based trees shallow.

5.4.1 Why B$^+$-trees are so widely used

B^+-trees and their variants are the preferred tree structures for external storage because they match the way disks and SSDs work:

- one node is typically one **page**,
- each internal node stores many separator keys and child pointers,
- high fanout means very small tree height,
- all leaves are linked in sorted order, which makes range scans efficient.

The broader family contains several variants, such as **B-tree**, B^+-tree, **B*-tree**, **B-link tree**, and **Bε-tree**. In this lecture, the main focus is the B^+-tree.

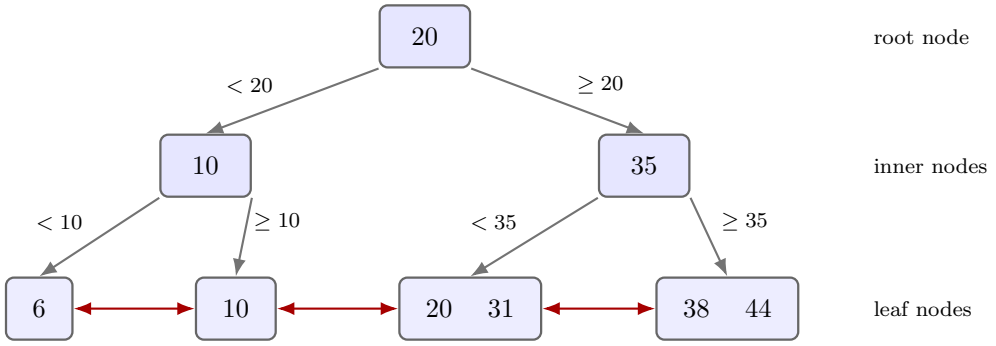
5.4.2 Core properties

A B^+-tree is an n-way search tree with the following properties:

- nodes come in three roles: **root**, **inner**, and **leaf**,
- the tree is **perfectly balanced**: every leaf is at the same depth,
- every node except the root is at least **half full**,
- an inner node has between $\lceil n/2 \rceil$ and n children,
- if an inner node has k children, it has $k-1$ separator keys,
- a leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ keys,
- the root is the one exception: it may contain fewer entries than ordinary internal nodes.

These conditions are exactly what keep the tree balanced and keep the height small.

In a B^+-tree:



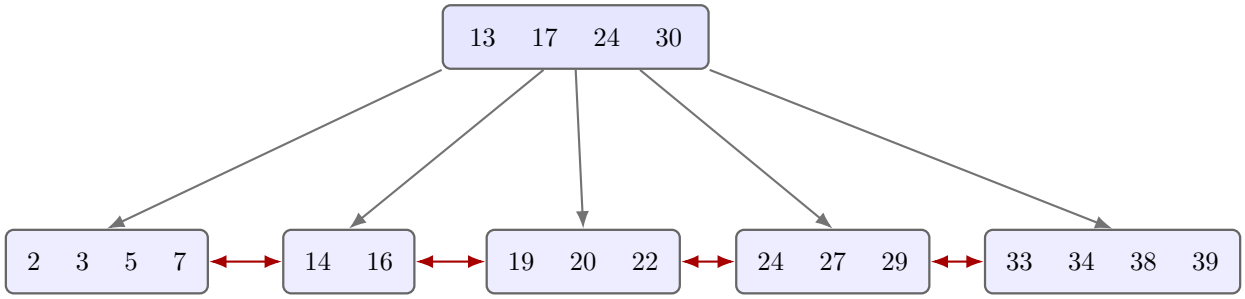
leaf links preserve key order for range scans

- **internal nodes** store only separator keys and child pointers,
- **leaf nodes** store the actual data entries,
- **sibling pointers** connect leaves from left to right, so after the first match the DBMS can keep scanning sequentially.

The data entry stored in each leaf can still follow Alternative 1, 2, or 3. The tree determines *how to find* the entry; the alternative determines *what the entry contains*.

5.4.3 How search works

Search always starts at the root. Key comparisons choose one child pointer at each internal node until the search reaches a leaf.



exact search stops in one leaf; range search continues through sibling pointers

The example above shows three typical cases:

Reading searches in a B+-tree

Query	Leaf reached	What we learn
`find 5`	$[2, 3, 5, 7]$	The key is present in that leaf.
`find 15`	$[14, 16]$	The key is not in the tree, because it would have to appear in that leaf between 14 and 16.
find keys ≥ 24	$[24, 27, 29]$	Start at the first matching leaf, then follow sibling pointers to the right.

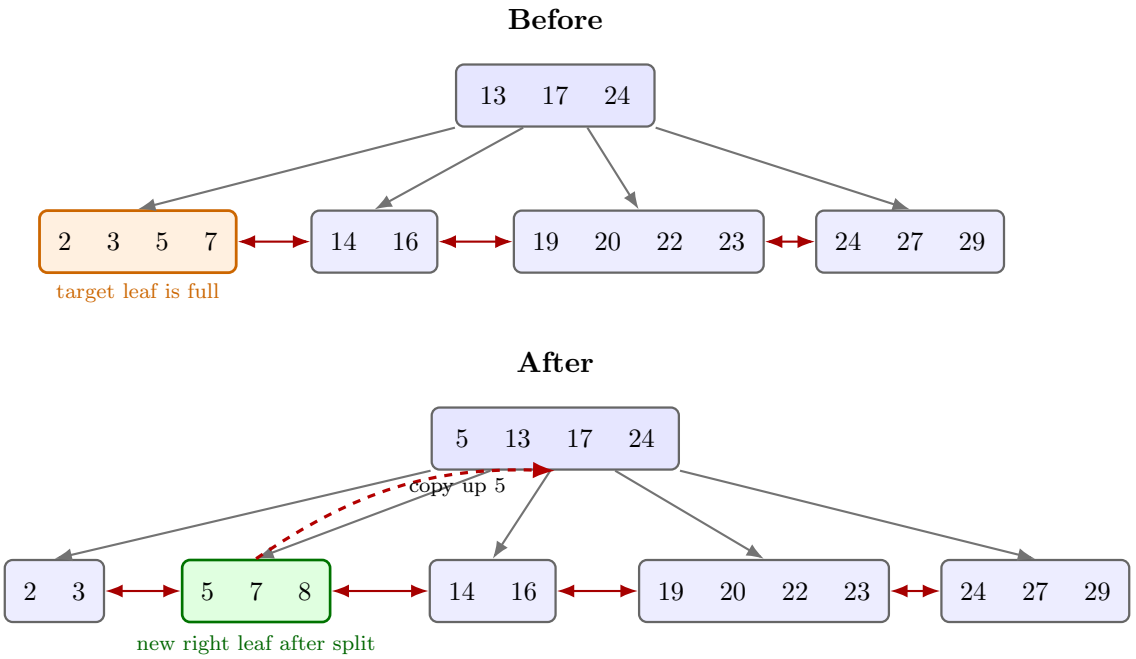
This is why B^+-trees support both:

- **equality search**: descend to one leaf and check it,
- **range search**: descend once, then scan leaf pages in order.

5.4.4 Insertion example: insert 8

Leaf split. If the target leaf is full, the DBMS allocates a new leaf and redistributes the entries. Then it inserts a separator key into the parent. If the parent also overflows, the split propagates upward.

Suppose we insert key 8 into the tree shown below.



The steps are:

- search from the root and reach the first leaf, because $8 < 13$,
- the target leaf already contains $[2, 3, 5, 7]$, so it is full,
- allocate one new leaf and redistribute the values evenly,
- after inserting 8, the two leaves become $[2, 3]$ and $[5, 7, 8]$,
- copy separator key 5 into the parent, because 5 is now the first key of the new right leaf,
- update sibling pointers so leaf-level scans still work left to right.

In this example, the parent still has room, so the insertion stops there. If the parent were also full, the same split idea would continue upward, possibly all the way to the root.

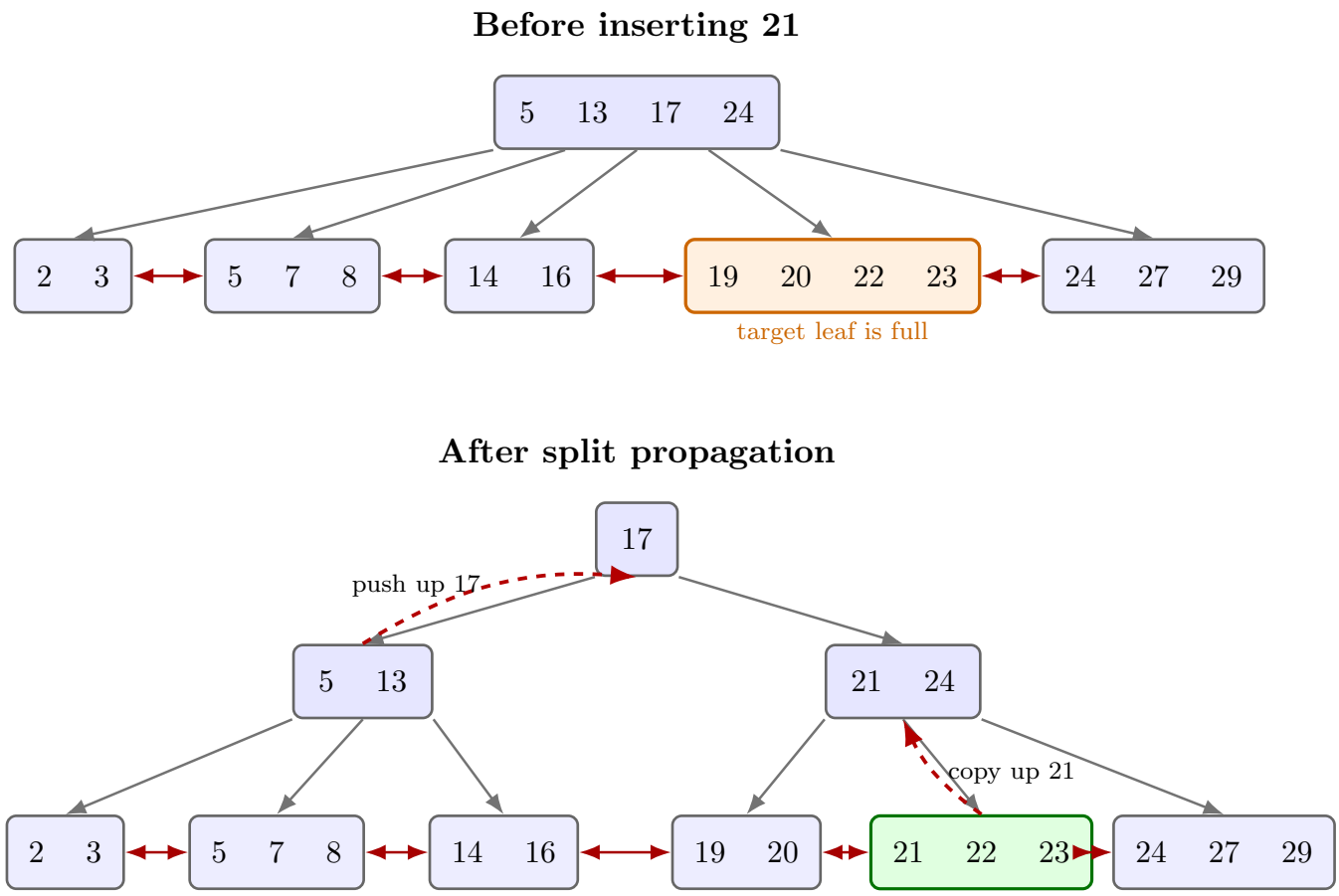
5.4.5 Insertion can propagate to the root

Root split. If an insertion reaches an overflowing root, the DBMS splits the root and creates a new root above it. This is the only case where the height of a B^+-tree increases.

Now insert key 21 into the current tree. The target leaf is $[19, 20, 22, 23]$, which is already full.

The sequence is:

- insert 21 into the full leaf $[19, 20, 22, 23]$,
- split that leaf into $[19, 20]$ and $[21, 22, 23]$,
- copy separator key 21 into the parent,
- the parent now overflows, so the parent must also split,
- push separator key 17 up to a brand-new root.



The final tree is taller by one level. This is the standard cascading-insertion case: a local split at the leaf causes another split above it.

In theory, redistributing entries differently could delay a root split. In practice, most implementations use the simple split rule because it is local, predictable, and easy to maintain.

5.4.6 Leaf split versus internal split

One detail is especially important: **leaf splits** and **internal splits** do *not* propagate keys in the same way.

Leaf split and internal split

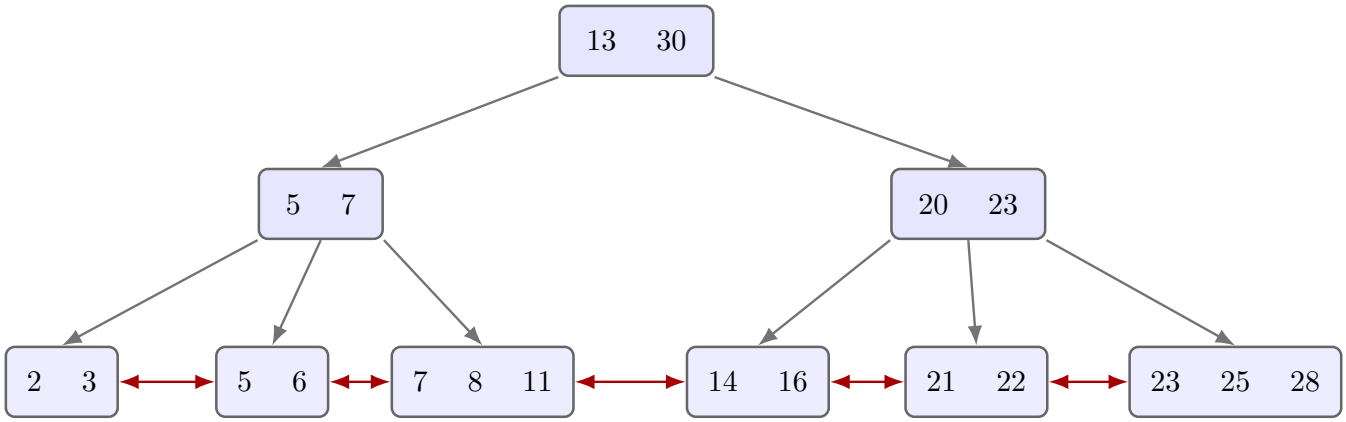
Case	What moves upward	Reason
Leaf split	separator key is **copied up**	The leaf stores actual data entries, so the key must remain in the leaf. The parent only needs a separator.
Internal split	middle separator is **pushed up**	Internal-node keys are only routing information, so the promoted separator appears only once, in the parent.

This is the clean rule to remember:

- **leaf split ⇒ copy up**,
- **internal split ⇒ push up**.

5.4.7 Another insertion sequence: 28, 6, and 25

The next example shows three insertions in a row. Some fit locally. Others cause a split and change the upper levels as well.



final tree after inserting 28, 6, and 25

Read the sequence like this:

- **insert 28**: the key fits into the rightmost leaf, so no split is needed,
- **insert 6**: the leaf $[5, 7, 8, 11]$ overflows, so it splits into $[5, 6]$ and $[7, 8, 11]$; key 7 is copied up,
- **insert 25**: the leaf $[21, 22, 23, 28]$ overflows, so it splits into $[21, 22]$ and $[23, 25, 28]$; key 23 is copied up,
- the parent then overflows, so an internal split pushes key 13 up and creates the final two-level tree shown above.

This example reinforces the main idea: insertions start locally at one leaf, but the structural effect may travel upward.

5.5 Deletion in B$^+$-trees

Deletion in a B$^+$-tree. Deletion removes a key from its leaf. If the leaf still satisfies the minimum-occupancy rule, the job is done. Otherwise the DBMS rebalances the tree by redistribution or merge.

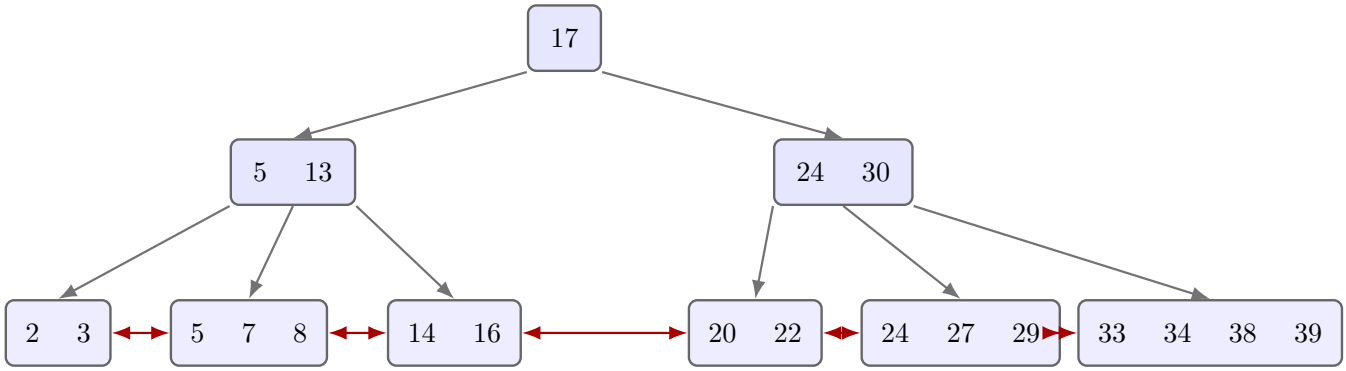
The standard deletion pattern is:

- start at the root and find the leaf L containing the entry,
- remove the entry from L,
- if L is still at least half full, stop,
- otherwise try **redistribution**: borrow an entry from a sibling with the same parent,
- if no sibling can spare an entry, **merge** L with one sibling,
- after a merge, delete the corresponding separator entry from the parent,
- if the parent underflows, apply the same logic again higher up,
- if the root ends with only one child, remove that root and decrease the tree height.

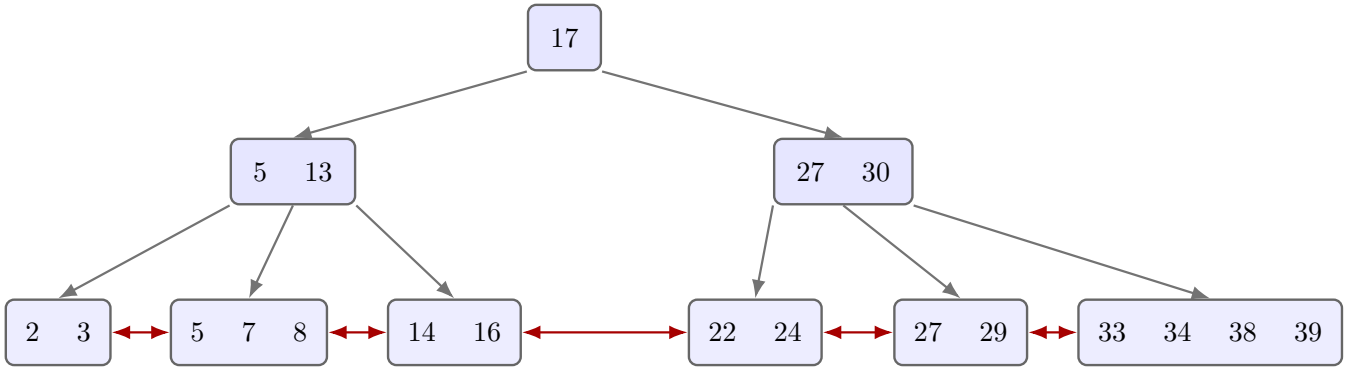
5.5.1 Worked example: delete 19 and 20

Deleting 19 is easy: after removing it, the leaf still has enough entries. Deleting 20 is more interesting, because the leaf becomes too small.

After deleting 19



After deleting 20 and redistributing



After deleting 19, the leaf becomes $[20, 22]$, which is still legal. After deleting 20, the same leaf becomes $[22]$, which violates minimum occupancy.

The DBMS therefore redistributes with the right sibling:

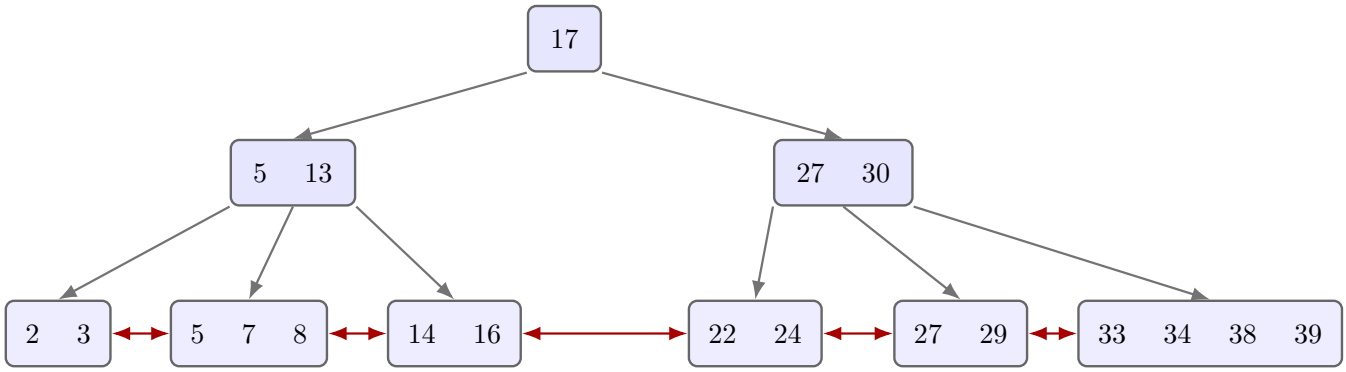
- the right sibling $[24, 27, 29]$ can spare one entry,
- the boundary moves, so the two leaves become $[22, 24]$ and $[27, 29]$,
- the parent separator is updated from 24 to 27.

In a leaf-level redistribution, the new boundary key is **copied up** to the parent, because leaves keep the actual data entries.

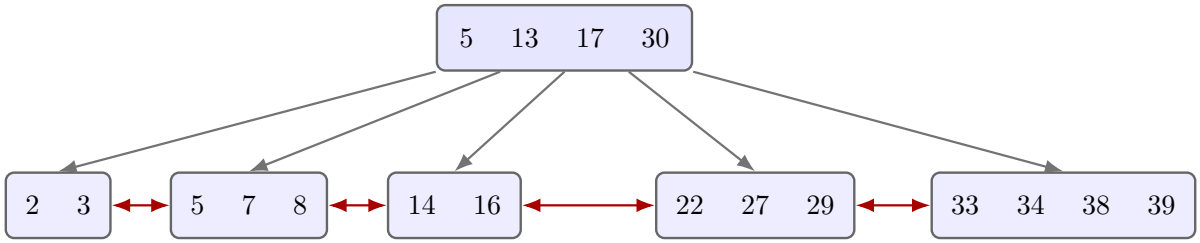
5.5.2 Worked example: delete 24 and shrink the tree

Now continue from the tree above and delete 24. This time redistribution is no longer enough, so the tree must merge and then shrink.

Before deleting 24



After merge and tree shrink



The logic is:

- deleting 24 turns leaf $[22, 24]$ into $[22]$, which is too small,
- its sibling $[27, 29]$ is already at minimum occupancy, so borrowing is impossible,
- the two leaves merge into one leaf $[22, 27, 29]$,
- the parent loses one separator and now underflows,
- the merge therefore propagates upward,
- the old root is removed, and the tree height decreases by one.

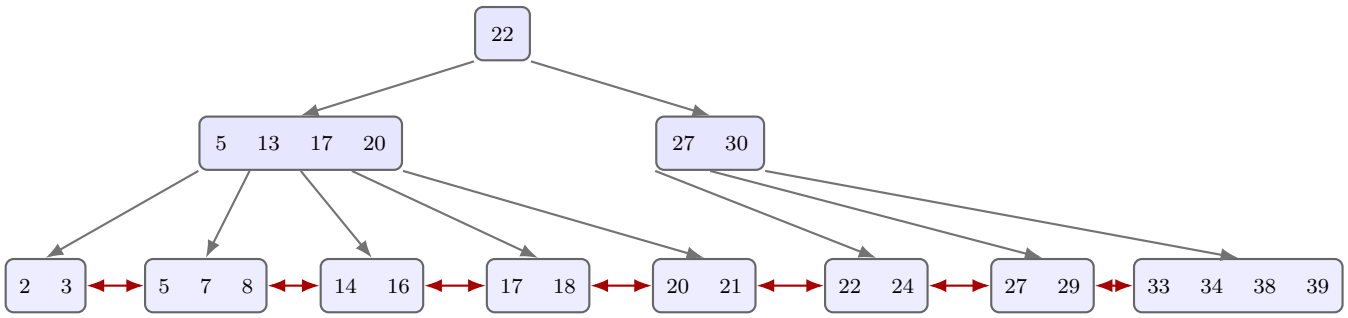
So deletion can do the opposite of insertion:

- insertion may **increase** the tree height,
- deletion may **decrease** the tree height.

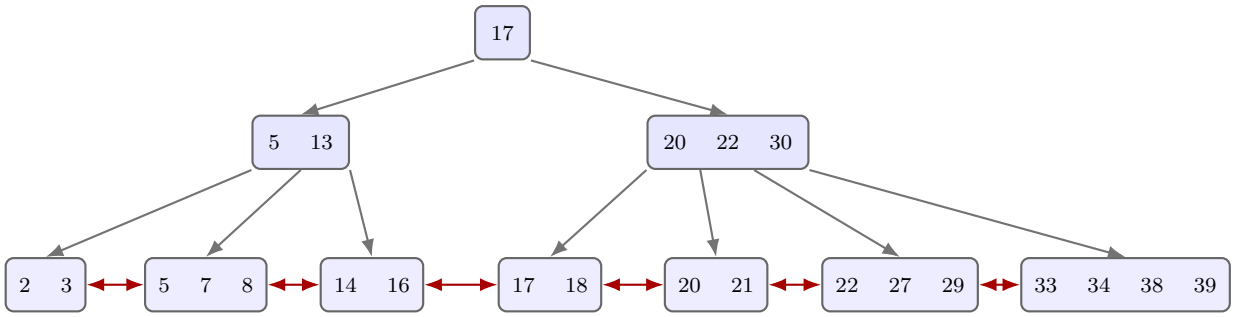
5.5.3 Redistribution at non-leaf levels

Redistribution is not only for leaves. It also works for internal nodes.

Before non-leaf redistribution



After pushing entries through the parent



The key idea is subtle but important:

- redistribution at the non-leaf level happens **through the parent**,
- one separator from the parent moves down into the underfull internal node,
- at the same time, one separator from the fuller sibling moves up into the parent,
- the corresponding child pointer moves with that separator.

So internal-node redistribution is not just "move one key sideways". It is really a **push-through-parent** operation.

5.6 Clustered indexes

Clustered index. An index whose logical order matches the physical order of the table data. Nearby index entries point to nearby records on disk.

The main idea is physical:

- the table is stored in the sort order of one chosen key, often the primary key,
- the storage may be **heap-organized but kept in that order**, or **index-organized**, where the leaf level stores the actual rows,
- because the data file can have only one physical order at a time, a table can have at most one clustered organization.

This is why clustered indexes are powerful for range scans:

- matching rows tend to sit on nearby pages,
- ordered scans can continue page by page with little random I/O.

But there is also a cost:

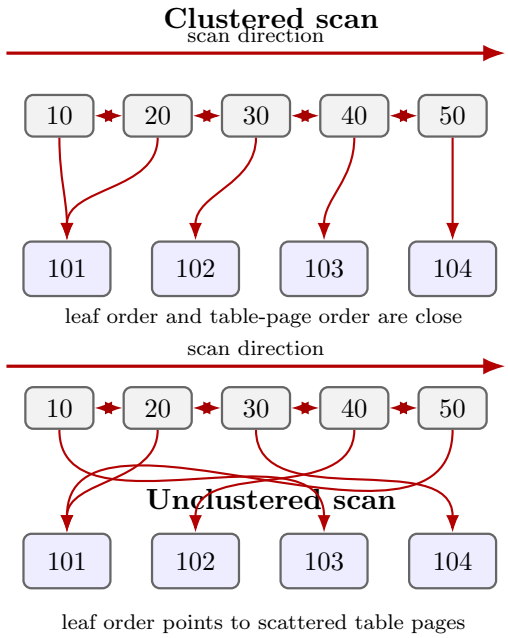
- inserts and deletes are harder, because maintaining physical order may force page splits, movement, or reorganization.

DBMSs differ here:

- some systems always organize a table around a clustered primary key; if no such key is declared, they create a hidden row identifier,
- other systems keep ordinary heap files and do not make the table itself follow one index order.

5.6.1 B^+-tree traversal and scan order

To scan many records in key order, the DBMS first descends to the leftmost relevant leaf and then follows leaf links from left to right.



The traversal itself is the same in both cases. The big difference is how the leaf entries map to table pages:

- with a **clustered** index, consecutive leaf entries usually point to nearby table pages, so scanning leaf after leaf also reads data pages in a mostly sequential order,
- with an **unclustered** index, consecutive leaf entries may point to tuples scattered over the table, so

following the leaf order can trigger many repeated or random page reads.

This explains why clustered tree indexes are excellent for ordered scans and large range queries.

For unclustered indexes, a common optimization is:

- first collect the matching RIDs or page IDs from the leaves,
- then sort or group them by **table page ID**,
- finally fetch table pages in page order.

This loses the natural key order during fetching, but it can save a lot of I/O when many tuples qualify.

5.7 B$^+$-tree design choices

Real systems all use the same core B^+-tree idea, but they still make many low-level design choices.

5.7.1 Node size

Node size is one of the most important choices because it controls both:

- **fanout**: larger nodes make the tree shallower,
- **intra-node cost**: larger nodes take longer to search once loaded.

As a rule of thumb, the slower the storage device, the larger the best node size tends to be:

Typical node-size tendencies

Storage medium	Typical size	Why
HDD	$\sim$ 1 MB	Large transfers amortize expensive seeks and rotational delay.
SSD	$\sim$ 10 KB	Random access is cheaper, so extremely large pages are less necessary.
In-memory	$\sim$ 512 B	Cache behavior matters more than disk I/O, so smaller nodes can be better.

These numbers are only rough guidelines. The best size still depends on the workload:

- long leaf scans benefit from larger nodes,
- repeated root-to-leaf lookups may prefer smaller nodes that fit caches better.

5.7.2 Merge threshold

The textbook rule says that nodes should stay at least half full. In practice, many DBMSs do *not* merge nodes immediately every time occupancy drops below that threshold.

Why delay merging?

- inserts are often more common than deletes, so a sparse page may soon fill again,
- eager merging causes extra page movement and separator updates,
- in practice, the average occupancy is often around 67%,
- some systems prefer periodic rebuilding over aggressive merge-on-delete.

So the theoretical half-full rule is best understood as a **guarantee for the ideal structure**, not always as a strict implementation policy after every update.

5.7.3 Variable-length keys

Variable-length search keys create another design problem: how do we store them inside fixed-size pages efficiently?

Common approaches include:

- **pointer-based keys**: store a pointer to the real attribute value instead of storing the full key inline,
- **variable-length nodes**: let the packed contents of one node vary in size and manage free space carefully,
- **padding**: extend each key to the maximum length of its type, which simplifies code but wastes space,
- **key indirection**: keep a small array of offsets or pointers that map to the actual key/value area inside the node.

The trade-off is the usual one:

- simpler layout usually wastes more space,
- compact layout saves space but makes memory management harder.

5.7.4 Intra-node search

Once one node is in memory, the DBMS still has to find the correct entry *inside* that node.

Searching inside one node

Method	Main idea	When it is attractive
Linear search	scan keys from left to right	good for small nodes; simple; often fast with SIMD/vectorized comparisons
Binary search	repeatedly compare with the middle key	good when nodes are larger and comparisons are cheap
Interpolation search	estimate the position from the key distribution	useful when keys are close to uniformly distributed

For interpolation search, one common estimate is:

$$\text{pos} \approx \text{low} + \frac{(x - \text{arr[low]})(\text{high} - \text{low})}{\text{arr[high]} - \text{arr[low]}},$$

where x is the search key.

The idea is to jump close to the expected location instead of always probing the middle. This works best when key values are spread fairly evenly.

5.8 Implementation View: Nodes and Lookup

5.8.1 What one node stores

At implementation level, a B^+-tree node is kept as a **sorted array** of entries inside one page.

The exact meaning of the value part depends on the node type:

- in an **internal node**, the value is a child page reference,
- in a **leaf node**, the value is a record pointer, a RID list, or the actual record itself, depending on the chosen data-entry alternative.

Leaf nodes also keep **prev** and **next** links to neighboring leaves. These are what make range scans efficient.

Even though the internal-node layout has one extra rightmost child pointer, the high-level implementation view is still simple:

- every node stores entries in **sorted key order**,
- search inside a node finds the first separator that tells us where to go next.

5.8.2 Lookup operation

The lookup algorithm follows the sorted structure of the tree:

1. Start at the root node.
2. If the current node is an internal node, search inside it for the first separator that determines the correct child.
3. Follow that child pointer to the next level.
4. Repeat until a leaf is reached.
5. In the leaf, either find the exact entry or determine the exact insertion position where the key would belong.

So a lookup can return either:

- the **matching entry**, if the key exists,
- or the **correct leaf position**, if the key does not exist but might be inserted later.

This lower-bound behavior is exactly what also makes insertions and range scans natural.

5.9 Concurrent Access to B$^+$-trees

Concurrency is harder in trees than in isolated heap pages because pages depend on one another structurally.

Simple page latching is not enough:

- it protects one page against concurrent modification,
- but a split or merge also changes parent-child relationships,
- so several pages may need to be synchronized together.

Lock coupling (latch crabbing). During a top-down traversal, the thread latches a parent, then latches the child it wants to enter, and only then releases the parent. The traversal therefore keeps a short overlapping chain of latches.

This classical method works well for search and for many simple updates:

- latch the root first,
- latch the next level,
- release the parent,
- continue downward in canonical root-to-leaf order.

Because latches are acquired in a fixed top-down order, ordinary traversals avoid deadlocks.

5.9.1 Why inserts are trickier

Insertion is harder than lookup because a split may propagate upward:

- a leaf split inserts a separator into the parent,
- that parent may also split,
- the split can continue all the way to the root.

This creates a problem for naive lock coupling: by the time the split reaches higher levels, those ancestors may already have been released.

5.9.2 Restart or optimistic coupling

A simple practical solution is:

1. first try the insertion using ordinary lock coupling,
2. if no split propagates through inner nodes, the operation succeeds normally,
3. otherwise release the latches,
4. restart the operation with a more conservative locking strategy that keeps the needed ancestors protected,
5. execute the update safely on the second pass.

This approach has a clear trade-off:

- it greatly reduces concurrency during the restarted operation,
- but the expensive case is rare,
- and the algorithm is much simpler to implement correctly.

5.10 B$^+$-trees in Practice

The main practical lesson is that B^+-trees are usually **very shallow**.

Many textbooks parameterize them with a value d, where an internal node has between d and $2d$ children. If we take a typical example with:

- $d = 100$,
- average fill factor $\approx 67\%$,

then the average fanout is roughly

$$2d \cdot 0.67 = 200 \cdot 0.67 \approx 134.$$

That already gives very large capacities:

Back-of-the-envelope capacities

Tree height	Approximate number of entries
3	$134^3 = 2{,}406{,}104$
4	$134^4 = 322{,}417{,}936$

This is why a tree of height 3 or 4 is often enough even for very large relations.

Top levels are also small enough to remain in memory:

Example memory footprint with 8 KB pages

Level	Size
level 1	1 page ≈ 8 KB
level 2	134 pages ≈ 1 MB
level 3	17,956 pages ≈ 140 MB

So in practice:

- the root is almost always memory-resident,
- often one or more upper levels are memory-resident too,
- only the lower levels require regular I/O.

Chapter 6

SQL-1 - SQL Introduction

6.1 Relational terminology

Although we already defined the relational model, let's align our vocabulary with SQL:

- **Relation** translates to a **table**.
- **Tuple** translates to a **row**.
- **Attribute** translates to a **column**.

Throughout this chapter, we use the following example `Students` table:

Students

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2
53777	White	white@cs	19	4.0

Primary Key (PK). The column (or group of columns) that uniquely identifies a row. Two rows **cannot** share the same PK. Above, `sid` is the primary key.

6.2 Basic SQL queries

6.2.1 The simplest query

To retrieve all contents of a table, use `SELECT *`:

```
SELECT * FROM Students S
```

`S` is an optional *alias* (range variable) for the `Students` table, useful when referencing columns. This returns the full table shown above.

6.2.2 Showing specific columns (Projection)

To retrieve only certain attributes, replace `*` with column names:

```
SELECT S.name, S.login FROM Students S
```

Projection. Selecting specific columns from a table. The query above projects `name` and `login` from `Students`.

name	login
Jones	jones@cs
Smith	smith@ee
White	white@cs

6.2.3 Showing specific rows (Selection)

To filter rows, use the `WHERE` clause:

```
SELECT * FROM Students S WHERE S.age = 18
```

Selection. Filtering rows based on a condition. The query above selects students whose age equals 18.

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2

6.2.4 SQL vocabulary

The general form of a basic SQL query:

```
SELECT [DISTINCT] target-list FROM relation-list WHERE qualification
```

- **relation-list**: a list of table names, each possibly followed by an alias.
- **qualification**: comparisons combined with `AND`, `OR`, `NOT`. Each comparison: `Attr` *op* `Const` or `Attr1` *op* `Attr2`.
- **target-list**: a list of attributes from the tables in *relation-list*.
- **DISTINCT**: optional keyword; eliminates duplicate rows. By default, SQL keeps duplicates (the result is a *multiset*).

6.2.5 Evaluation order

Conceptually, a SQL query is evaluated as follows:

1. **FROM**: compute the cross-product of all tables.
2. **WHERE**: discard tuples that fail the condition (*selection*).
3. **SELECT**: keep only the requested columns (*projection*).
4. If **DISTINCT** is specified, eliminate duplicate rows.

6.3 Querying multiple relations

To join two tables, list them both in `FROM` and express the join condition in `WHERE`. Consider the following `Enrolled` table alongside `Students`:

Enrolled

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
53666	History105	B

"Find all students with grade B":

```
SELECT S.name, E.cid
FROM Students S, Enrolled E
WHERE S.sid = E.sid AND E.grade = 'B'
```

S.name	E.cid
Jones	History105

6.3.1 Naïve evaluation in steps

The evaluation of the query above proceeds conceptually as follows:

Step 1 – Cross Product (FROM): Combine every row of `Students` with every row of `Enrolled`:

S.sid	S.name	S.login	S.age	S.gpa	E.sid	E.cid	E.grade
53666	Jones	jones@cs	18	3.4	53831	Carnatic101	C
53666	Jones	jones@cs	18	3.4	53831	Reggae203	B
53666	Jones	jones@cs	18	3.4	53650	Topology112	A
53666	Jones	jones@cs	18	3.4	53666	History105	B
53688	Smith	smith@ee	18	3.2	53831	Carnatic101	C
53688	Smith	smith@ee	18	3.2	53831	Reggae203	B
53688	Smith	smith@ee	18	3.2	53650	Topology112	A
53688	Smith	smith@ee	18	3.2	53666	History105	B

Step 2 – Apply the predicate (WHERE): Keep only rows where `S.sid = E.sid` **and** `E.grade = 'B'`. Only one row survives: Jones (sid 53666) enrolled in History105 with grade B.

Step 3 – Project the requested columns (SELECT): Keep only `S.name` and `E.cid`, yielding the final result: `(Jones, History105)`.

Running example: Sailors, Boats, Reserves

The following three relations are used throughout the rest of this chapter.

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
95	Bob	3	63.5

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Reserves

sid	bid	day
22	101	10/10/96
95	103	11/12/96

6.4 Aggregate operators

Aggregate function. A function that operates on a set (or multiset) of values from a column and returns a **single** summary value. Aggregates are a significant extension of relational algebra.

Function	Description
`COUNT(*)`	number of rows
`COUNT([DISTINCT] A)`	number of (distinct) non-null values in `A`
`SUM([DISTINCT] A)`	sum of (distinct) values
`AVG([DISTINCT] A)`	average of (distinct) values
`MAX(A)`	maximum value
`MIN(A)`	minimum value

6.4.1 Examples

Find the number of sailors:

```
SELECT COUNT(*) FROM Sailors S
```

Find the average age of sailors with rating 10:

```
SELECT AVG(S.age) FROM Sailors S WHERE S.rating = 10
```

Count distinct ratings of sailors named Bob:

```
SELECT COUNT(DISTINCT S.rating) FROM Sailors S WHERE S.sname = 'Bob'
```

6.4.2 Name and age of the oldest sailor(s)

This example illustrates a common pitfall with aggregates.

Incorrect query

```
SELECT S.sname, MAX(S.age) FROM Sailors S
```

This is **wrong**: `MAX` returns a single value over the entire column, so there is no meaningful way to pair it with a particular `sname`.

Correct approach — subquery

```
SELECT S.sname, S.age
FROM Sailors S
WHERE S.age = (SELECT MAX(S2.age) FROM Sailors S2)
```

The inner query computes the maximum age; the outer query retrieves every sailor matching that value.

Scalar subquery. A subquery that returns exactly one row and one column. It can appear wherever a single value is expected (e.g. in a `WHERE` comparison).

An equivalent alternative places the subquery on the left:

```
SELECT S.sname, S.age
FROM Sailors S
WHERE (SELECT MAX(S2.age) FROM Sailors S2) = S.age
```

6.5 GROUP BY

So far, aggregates have been applied to *all* qualifying tuples at once. Sometimes we need to apply them to each of several *groups* independently.

Consider: find the age of the youngest sailor for each rating level.

- We don't know in advance how many rating levels exist or what their values are.
- If ratings range from 1 to 10, we *could* write 10 separate queries:

For $i = 1, 2, \ldots, 10$:

```
SELECT MIN(S.age) FROM Sailors S WHERE S.rating = i
```

This is obviously impractical.

GROUP BY clause. Partitions tuples into groups based on equal values of the specified columns, then applies aggregate functions independently to each group.

6.5.1 Using GROUP BY

We use an extended `Sailors` table for the following examples:

Sailors (extended)

sid	sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
95	Bob	3	63.5
52	Smith	3	74.5
68	John	7	56.5
81	Ana	7	71.5

"Find the youngest age for each rating level":

```
SELECT S.rating, MIN(S.age) FROM Sailors S GROUP BY S.rating
```

The tuples are first partitioned by `rating` (groups: rating 3, rating 7, rating 8), then `MIN(S.age)` is evaluated within each group:

rating	MIN(age)
3	63.5
7	45.0
8	55.5

GROUP BY semantics. The `SELECT` clause may only contain:

- attributes listed in the `GROUP BY` clause, and
- aggregate functions applied to other attributes.

Any non-aggregated column in `SELECT` that is *not* in `GROUP BY` is an error.

6.5.2 HAVING — filtering groups

`HAVING` filters *groups* (after grouping), just as `WHERE` filters *individual tuples* (before grouping).

"Find the youngest age for each rating level that has **at least two** sailors":

```
SELECT S.rating, MIN(S.age)
FROM Sailors S
GROUP BY S.rating
HAVING COUNT(*) > 1
```

Groups with only one member (rating 8, containing only Lubber) are eliminated:

rating	MIN(age)
3	63.5
7	45.0

HAVING clause. A filter applied *after* grouping. It uses aggregate conditions to keep or discard entire groups, unlike `WHERE` which operates on individual rows before any grouping.